VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi - 590 018, Karnataka

Capstone Project Phase – 1

(Course code: 24AM6PWPW1)

PROJECT WORK REPORT

AI-Driven Analysis and Optimization of CI/CD Pipelines for Build Failure Prediction

In the Department of

Machine Learning

(UG Program: B.E. in Artificial Intelligence and Machine Learning)

Submitted by

Pakam Yasaswini	**1BM23AI121**
Reem Rashid	**1BM23AI148**
Sarayu KS	**1BM23AI171**

6th Semester

Under the Guidance of

Dr. Rajeshwari M.R.

Designation

Dept. of MEL, BMSCE, Bengaluru – 19

DEPARTMENT OF MACHINE LEARNING

B.M.S. COLLEGE OF ENGINEERING

(An Autonomous Institute, Affiliated to VTU)

P.O. Box No. 1908, Bull Temple Road, Bengaluru - 560 019

B.M.S COLLEGE OF ENGINEERING

(An Autonomous Institute, Affiliated to VTU)

P.O. Box No. 1908, Bull Temple Road, Bengaluru - 560 019

DEPARTMENT OF MACHINE LEARNING

CERTIFICATE

This is to certify that Ms. Pakam Yasaswini bearing USN: 1BM23AI121, Ms. Reem Rashid bearing USN: 1BM23AI148, and Ms. Sarayu K S bearing USN: 1BM23AI171 have satisfactorily completed the Course – Capstone Project Work (Course Code: 24AM6PWPW1) with the title **"AI-Driven Analysis and Optimization of CI/CD Pipelines for Build Failure Prediction"** in partial fulfillment of academic curriculum requirements of the 6th semester UG Program – B. E. in Artificial Intelligence and Machine Learning in the Department of Machine Learning, BMSCE, an Autonomous Institute, affiliated to Visvesvaraya Technological University, Belagavi during May 2026. It is also stated that the base work considered for the presentation is used only for academic purpose and not used in its original form anywhere for award of any degree.

Student Signature

Signature of the Faculty,

Dr. Rajeshwari M.R.

Designation, Dept. of MEL, BMSCE

Signature of the Head

Dr. M Dakshayini

Prof. & Head, Dept. of MEL, BMSCE

External Examination

Examiner Name and Signature

1.

2.

DECLARATION

We, Ms. Pakam Yasaswini bearing USN: 1BM23AI121, Ms. Reem Rashid bearing USN: 1BM23AI148, and Ms. Sarayu K S bearing USN: 1BM23AI171, hereby declare that the project titled "AI-Driven Analysis and Optimization of CI/CD Pipelines for Build Failure Prediction" is an original work carried out by us under the guidance of Dr. Rajeshwari M.R., Assistant Professor, at BMSCE, Bengaluru. The work presented in this report has not been submitted elsewhere for any other degree, diploma, or certification.

We declare that all the references and sources used in the report have been properly acknowledged and cited. No part of this report has been copied, plagiarized, or reproduced from another work without proper attribution.

Pakam Yasaswini – (USN: 1BM23AI121)

Reem Rashid – (USN: 1BM23AI148)

Sarayu KS – (USN: 1BM23AI171)

ACKNOWLEDGEMENT

We would like to express our deepest gratitude and sincere thanks to our guide Dr. Rajeshwari M.R., Assistant Professor, Department of Machine Learning, BMSCE, Bengaluru, for her keen interest, continuous guidance, and encouragement in bringing this project work to completion.

We are grateful to Dr. M. Dakshayini, Professor and Head, Department of Machine Learning, Bengaluru, for her valuable suggestions and support throughout the project work.

We express our profound gratitude to the Hon'ble Management and Dr. Bheemsha Arya, Principal, BMSCE, Bengaluru, for providing the necessary facilities and a supportive environment to carry out this project successfully.

We would also like to thank all the staff members of the Department of Machine Learning for their support, encouragement, and guidance during the course of this project.

Last but not the least, we would like to thank our parents, family, and friends whose constant encouragement and support made this work possible.

Pakam Yasaswini – (USN: 1BM23AI121)

Reem Rashid – (USN: 1BM23AI148)

Sarayu KS – (USN: 1BM23AI171)

ABSTRACT

Continuous integration and continuous deployment (CI/CD) pipelines are central to modern software development, but failed builds still interrupt developers, delay releases and waste compute. CI/CD platforms automate and monitor pipelines; they offer little help in predicting which builds will fail, diagnosing why, or spotting slow stages.

This project builds and evaluates an AI-driven system that predicts build failures *before* a build runs, detects slow or stalled pipeline jobs, groups failure logs to suggest root causes, and presents the results on an interactive dashboard. The predictor was developed on 858,368 Travis CI builds from 947 open-source projects and evaluated on 146,556 GitHub Actions runs from 60 further projects.

A central finding is that the usual way of measuring these models greatly overstates them. With a random train/test split, classifiers such as Random Forest, Decision Tree and SVM appear strong (AUC up to 0.91), but they quietly use future information. Under a strict protocol, the same task scores 0.63. That protocol uses only information available when a build is triggered, counts a past build only once it has finished, evaluates chronologically, and measures how well a model ranks *each repository's own* builds. Within this protocol, our final model (Tier E) adds the history of the build's own branch or pull request, the state of builds still running, and a learning-to-rank objective. It improves within-repository AUC from 0.628 to 0.698 on Travis CI, including on builds that break a previously passing branch (0.556 to 0.622).

We then tested the frozen model on GitHub Actions with protocols fixed and hashed before collecting any data. The first test (30 repositories) improved in every fold but was inconclusive under the pre-registered rule. A confirmation on 30 new repositories met the rule in all five folds (0.911 vs. 0.763).

The deployed system receives GitHub webhooks and scores each workflow run when it is requested. Each repository's riskiest 15% of recent runs are flagged as alerts, which catches about 41% of a repository's failures. A read-only public demonstration and a pre-registered forward-looking test of the frozen system complete the work.

TABLE OF CONTENTS

Declaration		**i**
Acknowledgement		**ii**
Abstract		**iii**
Table of Contents		**iv**
List of Figures		**vi**
List of Tables		**vii**
1	**INTRODUCTION**	**1**
	1.1 Overview	1
	1.2 Summary of the Work Carried Out	3
	1.3 Organization of the Report	4
2	**PROBLEM STATEMENT**	**5**
	2.1 Problem Statement	5
	2.2 Motivation	5
	2.3 Objectives and Scope	6
3	**LITERATURE SURVEY**	**7**
	3.1 Existing Systems	7
	3.2 Research on Build Failure Prediction	7
	3.3 Research Gap	10
4	**SYSTEM REQUIREMENT SPECIFICATION**	**12**
	4.1 Functional Requirements	12
	4.2 Non-functional Requirements	12
	4.3 Hardware Requirements	13
	4.4 Software Requirements	13
5	**SYSTEM DESIGN**	**15**
	5.1 System Architecture	15
	5.2 Proposed Methodology	16
	5.3 Detailed Design	18
	5.3.1 Class Diagram	18
	5.3.2 Activity Diagram	19

	5.3.3 Use Case Diagram	20
6	**IDENTIFICATION OF TOOLS AND TECHNIQUES**	**22**
	6.1 Development Tools	22
	6.2 CI/CD and Integration Tools	22
	6.3 Visualization and Deployment Tools	23
	6.4 Techniques Used	23
7	**IMPLEMENTATION**	**26**
	7.1 Datasets	26
	7.2 Leakage-Safe Data Pipeline	27
	7.3 Feature Engineering	27
	7.4 Prediction Models	28
	7.5 Anomaly, Bottleneck and Root Cause Modules	29
	7.6 Deployed System	29
	7.7 Dashboard	30
	7.8 Testing and Requirement Traceability	33
8	**RESULTS AND DISCUSSION**	**35**
	8.1 Evaluation Protocol	35
	8.2 How Much the Usual Evaluation Overstates Results	35
	8.3 Results on Travis CI	37
	8.4 New Breakages	40
	8.5 Pre-Registered Tests on GitHub Actions	40
	8.6 Deployed Model and Alert Policy	42
	8.7 Negative Results	44
	8.8 Threats to Validity	44
9	**CONCLUSION AND FUTURE WORK**	**46**
10	**REFERENCES**	**48**

LIST OF FIGURES

Figure No.	Description	Page
Fig 1	Overview of the AI-driven CI/CD optimization system	2
Fig 2	System architecture of the proposed CI/CD optimization framework	15
Fig 3	Proposed methodology for AI-driven CI/CD pipeline optimization	16
Fig 4	Class diagram of the CI/CD optimization system	18
Fig 5	Activity diagram of the CI/CD optimization system	19
Fig 6	Use case diagram of the CI/CD optimization system	20
Fig 7	Log clustering flow for root cause analysis	25
Fig 8	Scoring path of the deployed system	29
Fig 9	Dashboard: summary cards and daily failure-rate trend	31
Fig 10	Pipelines page with recent-risk sparklines	31
Fig 11	Risk timeline of a repository with its alert threshold	32
Fig 12	Submit Build page: scoring a run with server-derived history	32
Fig 13	What-If page with SHAP explanation	33
Fig 14	Development comparison of six classifiers (random split)	36
Fig 15	AUC under progressively stricter evaluation	37
Fig 16	Within-repository AUC of Tier C and Tier E across all evaluations	38
Fig 17	Gain of Tier E as outcomes are withheld longer	39
Fig 18	Per-fold gain with 95% confidence intervals	41
Fig 19	Tier E vs. Tier C vs. a naive persistence score	42
Fig 20	Global threshold vs. per-repository alert budget	43
Fig 21	Dashboard: risk distribution and model-evidence panel	44

LIST OF TABLES

Table No.	Description	Page
3.1	Literature survey summary	9
4.1	Hardware requirements	13
4.2	Software requirements	13
7.1	Datasets used in the project	26
7.2	Feature tiers	27
7.3	Requirement-to-implementation traceability	33
8.1	Development comparison of classifiers (random split)	36
8.2	The same task under progressively stricter evaluation	37
8.3	Travis CI results by finish buffer	38
8.4	Attribution of the Tier E gain on Travis CI	39
8.5	First GitHub Actions test	40
8.6	GitHub Actions confirmation test	41
8.7	Deployed model on its holdout	42
8.8	Alert policies compared	43

CHAPTER 1 – INTRODUCTION

1.1 Overview

Introduction to CI/CD Pipelines

The implementation of Continuous Integration and Continuous Deployment (CI/CD) pipelines has become crucial in the modern era of software development and DevOps engineering. Modern software development requires rapid application delivery without compromising software quality and reliability. Traditional ways of software integration and testing can no longer cope with the increasing complexity and size of contemporary applications. For this reason, CI/CD pipelines have been widely adopted to automate software integration, testing, and deployment [11].

Continuous Integration is the practice in which developers continuously merge changes into a shared repository. Whenever the code is updated, automated builds and tests run to check its correctness. In the Continuous Delivery stage, changes that pass testing are automatically deployed to a staging or production environment.

Existing CI/CD Platforms

A variety of CI/CD tools including Jenkins, GitHub Actions, GitLab CI/CD, Travis CI, CircleCI, and Azure DevOps are extensively used in the software industry. They automate stages of the software development life cycle such as source code integration, dependency management, building, testing, packaging, deployment, and monitoring. A pipeline can have many stages, executed serially or concurrently depending on project needs. The automation these tools offer helps detect problems early and improves the deployment process.

These tools also offer dashboards, logs and monitoring, which help developers manage software delivery more effectively.

Challenges in Existing CI/CD Systems

Despite these benefits, CI/CD pipelines still face difficulties. Large-scale studies of CI histories report that a substantial share of builds fail [4, 19], for reasons ranging from test failures, dependency conflicts and wrong configurations to compilation errors, flaky tests and unstable execution environments. In the data used in this project, 19.5% of Travis CI builds and between 8.9% and 28.2% of GitHub Actions runs failed. Build failures disrupt development, delay releases, and reduce developer productivity.

The manual effort needed to debug failed builds is one of the most prominent problems in operating CI/CD pipelines. To fix a failure, a developer must read through execution logs to find

out what went wrong. Pipelines usually have several stages and many components, so this can take a significant amount of time and adds to developer workload.

Apart from failures, CI/CD pipelines suffer from performance inefficiencies such as excessive execution time, redundant testing, poor stage scheduling and wasted resources [6]. Some stages take unacceptably long because of dependency installation issues, testing problems, or infrastructure issues.

Role of Artificial Intelligence and Machine Learning

Advances in Artificial Intelligence (AI), Machine Learning (ML) and data analytics make it possible to manage CI/CD pipelines more predictively. Machine learning algorithms can learn from past build data to recognize patterns associated with failed builds and anomalous pipeline behavior, helping developers anticipate problems before they occur.

A number of research papers use machine learning techniques such as Random Forest, Decision Trees, Support Vector Machines (SVM), neural networks and deep learning to predict CI build failures [2, 7, 9]. Others consider anomaly detection, automated log analysis and clustering [1]. However, each of these studies solves a specific part of the problem, and few integrate them into a complete pipeline-analysis solution. Moreover, as this project shows, the way such predictors are usually evaluated can greatly overstate how well they work in practice.

Proposed System Overview

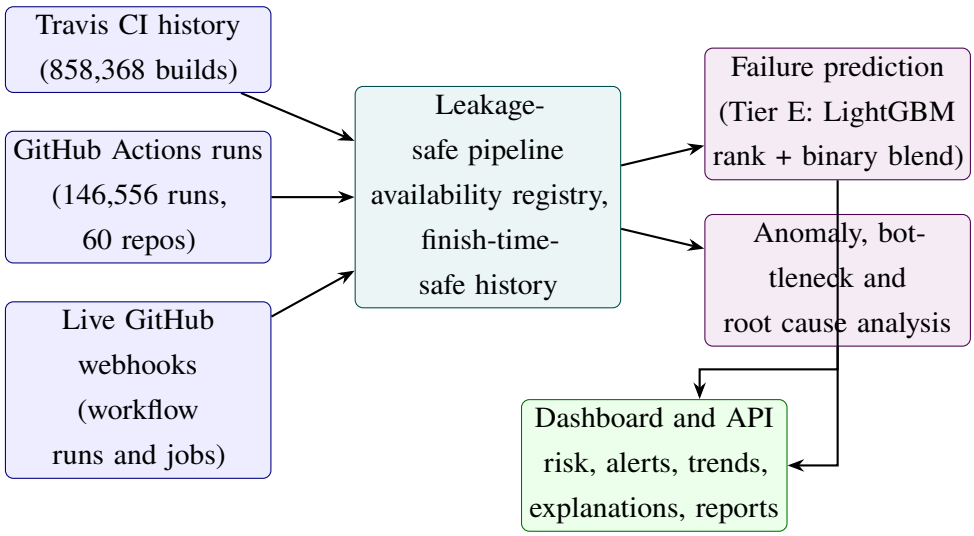


Fig 1: Overview of the AI-driven CI/CD optimization system

The project "AI-Driven Analysis and Optimization of CI/CD Pipelines for Build Failure Prediction" addresses these constraints with an intelligent system that predicts build failures and supports pipeline optimization (Fig 1). It combines machine learning, anomaly detection and log analysis

to predict build failures before a build runs, recognize slow or stalled pipeline jobs, group recurring failure patterns, and suggest remedies.

The system learns from historical CI/CD data: build outcomes, commits and the size of each change, timing, and the history of each repository, branch and pull request. It was developed on the TravisTorrent dataset [12] of Travis CI builds and validated on GitHub Actions data that we collected. Random Forest, Decision Tree and Support Vector Machine models from the original plan were trained and compared. The final model is a LightGBM learning-to-rank ensemble that uses the history of the build's own branch or pull request. Anomaly detection identifies jobs that run abnormally long, and log clustering and rule-based diagnosis group similar failures.

Significance of the Project

The platform helps teams spend a limited review budget where it matters most: it flags the riskiest builds before they run, explains each prediction, and highlights recurring failure types and slow jobs. This can reduce debugging effort and wasted compute and make pipelines more reliable. A visualization dashboard lets developers and administrators observe predictions, failure trends, bottlenecks and suggestions interactively.

The project is significant for software engineering and DevOps for a second reason. It shows how AI methods can support CI/CD processes, and it also shows how such methods must be evaluated before their numbers can be trusted.

1.2 Summary of the Work Carried Out

The project progressed beyond the design proposed in Phase 1 to a complete, evaluated and deployed system:

- **Leakage-safe data pipeline.** A registry records when every input becomes available and blocks any input that would only be known after a build finishes. History features count a previous build only once it has finished.
- **Prediction models.** Random Forest, Decision Tree, SVM, XGBoost, LightGBM and a stacking ensemble were compared, and a new model (Tier E) was developed that uses the history of each branch or pull request.
- **Rigorous evaluation.** Models were evaluated chronologically and *within each repository*, with confidence intervals, and tested on GitHub Actions under protocols written and fixed before the data was collected.
- **Deployed system.** A FastAPI service scores GitHub Actions runs automatically through webhooks, issues alerts under a per-repository review budget, detects job bottlenecks and diagnoses failure logs. A React dashboard presents the results, and a read-only public

demonstration is available online.

1.3 Organization of the Report

This report is organized into ten chapters. Chapter 1 introduces the project and its significance. Chapter 2 presents the problem statement, motivation, objectives and scope. Chapter 3 reviews existing systems, related research and research gaps. Chapter 4 outlines the functional, non-functional, hardware and software requirements. Chapter 5 describes the system design, architecture, methodology and UML diagrams. Chapter 6 discusses the tools and techniques used. Chapter 7 describes the implementation: datasets, data pipeline, features, models, the deployed system and the dashboard. Chapter 8 presents and discusses the results. Chapter 9 concludes the report and outlines future work. Chapter 10 lists the references used in the study.

CHAPTER 2 – PROBLEM STATEMENT

2.1 Problem Statement

Software development today relies heavily on CI/CD pipelines that automate building, testing and deployment. Despite the efficiency and speed they bring, CI/CD pipelines suffer from frequent failures caused by test failures, dependency issues, configuration mismatches, compilation problems, flaky tests and execution instability, which delay software delivery and reduce developer productivity. CI/CD tools such as Jenkins, GitHub Actions and GitLab CI/CD concentrate on executing and observing pipelines. They do not predict likely failures, identify root causes, find bottlenecks, or recommend improvements. Developers must investigate long execution logs to find problems. Moreover, published failure predictors are usually evaluated in ways that leak future information, so their reported accuracy is not what a team would experience. It is therefore reasonable to develop an artificial intelligence-based solution that analyzes CI/CD execution history and predicts failures before a build runs, under an evaluation that matches real use. The same solution should detect abnormal executions, group failure cases, and generate optimization suggestions.

2.2 Motivation

CI/CD pipelines are one of the most important aspects of today's DevOps practice because they automate software delivery. While automation increases developer productivity, pipeline failures pose significant challenges. Frequent failures disrupt development work, consume unnecessary computational resources, and delay the delivery of new software.

Finding the error behind a failed pipeline means manually analyzing execution logs, which becomes harder when pipelines have many steps with dependencies between them. Today's CI/CD services focus on automation and monitoring and offer little intelligent analytics.

A second motivation emerged during the project. Build failure prediction looks far easier than it is when it is evaluated carelessly: on our data, the same kind of model scores 0.91 AUC on a random split but only 0.63 when evaluated the way a team would use it (Chapter 8). A useful system must therefore be both effective and honestly measured.

This project explores an artificial intelligence-powered CI/CD optimization platform that predicts failures, discovers inefficient pipeline steps, analyzes failure patterns and generates recommendations, all evaluated under a protocol that reflects real use.

2.3 Objectives and Scope

Objectives

1. Develop a machine learning-based solution that predicts CI/CD pipeline build failures from historical data before the build runs, and evaluate it with a leakage-safe, chronological, within-repository protocol.

2. Detect bottlenecks and abnormal execution behavior in CI/CD pipeline jobs to support pipeline optimization.

3. Provide automated root cause analysis and optimization suggestions to minimize developer debugging effort and improve reliability, presented through an interactive dashboard.

Scope

This study uses historical CI/CD build data to predict build failures and analyze performance issues. The data covers 858,368 Travis CI builds from 947 projects (2011–2016) and 146,556 GitHub Actions runs from 60 projects (2025–2026). Machine learning is used to predict failures, and anomaly detection and log analysis are used to find bottlenecks and group failures. The findings are delivered as predictive analytics, bottleneck detection, root cause analysis and optimization suggestions, through an API that receives live GitHub Actions events and a web dashboard. The system is advisory: it never blocks or changes CI builds.

Impact

For industry, the system helps teams prioritize review and debugging effort and reduce wasted builds. For academia, it provides an evaluation protocol and pre-registered evidence that make build failure prediction results more trustworthy and comparable.

CHAPTER 3 – LITERATURE SURVEY

3.1 Existing Systems

Continuous Integration and Continuous Deployment (CI/CD) systems are widely used in modern software development to automate software building, testing and deployment [11]. They help development teams deliver software faster, reduce manual effort and maintain continuous integration workflows. Popular CI/CD platforms such as Jenkins, GitHub Actions, GitLab CI/CD and Travis CI are used across the software industry.

Jenkins is one of the most widely used open-source CI/CD automation tools. It supports automated builds, testing and deployment through customizable pipelines and plugins. Jenkins automates software delivery efficiently but focuses on execution and monitoring; it cannot predict failures or identify root causes automatically.

GitHub Actions is a cloud-based CI/CD platform integrated directly with GitHub repositories. It automates workflows using YAML configuration files. It improves automation, but developers still inspect logs manually when failures occur, because the platform has no predictive analytics or optimization features.

GitLab CI/CD provides integrated DevOps functionality, including automated testing, deployment, monitoring and repository management, within one platform. It supports scalable delivery workflows but focuses on automation and execution management rather than intelligent prediction and optimization.

Travis CI is commonly used in open-source projects for automated builds and tests. It supports many programming languages and integrates easily with GitHub. However, it offers limited analytics, and failed builds must be debugged manually. Its build histories, published as the TravisTorrent dataset [12], are the most widely used resource for CI research and are the main development data of this project.

Monitoring and visualization tools such as **Grafana, Kibana and the ELK Stack** are also widely used to analyze CI/CD logs and execution metrics. They help developers track pipeline performance and visualize trends, but they rely on manual interpretation to identify issues and inefficiencies.

3.2 Research on Build Failure Prediction

Many studies have explored Artificial Intelligence and Machine Learning techniques for CI/CD pipeline management and build failure prediction.

Empirical studies of CI and its failures

Hilton et al. [11] studied the usage, costs and benefits of CI in open-source projects and found that CI helps projects release more often but has real costs. Rausch et al. [4] conducted a large-scale empirical study of build failures in open-source Java projects and found that test failures cause the majority of broken builds. The study provided useful statistical insights but did not include predictive or optimization mechanisms. Durieux et al. [19] analyzed more than 35 million Travis CI jobs, and later [5] studied restarted and flaky builds, identifying unstable tests and network problems as common causes. Gallaba et al. [18] showed that historical build data contains noise and heterogeneity, such as failures that are ignored or builds that are restarted, that can mislead analyses. Ghaleb et al. [6] studied why some CI builds take a long time, which motivates bottleneck detection.

Build outcome prediction

Hassan and Wang [13] proposed a change-aware build prediction model that uses features of the code change to predict build outcomes and avoid stalls. Ni and Li [14] used cascaded classifiers to predict build outcomes cost-effectively, and Xia and Li [15] compared machine learning classifiers for predicting CI build results. Saidani et al. [2] used Multi-Objective Genetic Programming on historical build information and software metrics. The study achieved good prediction accuracy but provided no automated root cause analysis or optimization support. Saidani et al. [7] later applied deep learning models such as LSTM and CNN and improved accuracy over traditional models. However, these models need high computational resources and explain their predictions poorly. Chen et al. proposed BuildFast [16], a history-aware predictor that uses the outcomes of previous builds, and Jin and Servant [17] studied cost-efficient approaches that decide which builds to run. Barrak et al. [3] replicated a study of why builds fail using Random Forest models, identifying code complexity, build history and developer activity as important indicators. The system performed poorly on imbalanced data and focused on Java projects. Kumar et al. [9] evaluated models such as Support Vector Machines for predicting build failures from historical pipeline data. Their results were promising, but the work lacked automated root cause analysis and bottleneck detection.

Diagnosis, triage and practice

Zhang et al. proposed BuildSheriff [1], a change-aware triage system that uses stack-trace similarity and code-change analysis to group related test failures and reduce debugging effort. However, the approach mainly supports Java projects in Travis CI environments and struggles with flaky tests. Hong et al. [10] studied practitioners' challenges with, and perceptions of, build failure prediction at Atlassian. They highlight that developers need predictions that are trustworthy and actionable in their own projects. Farihane [8] presented a systematic mapping study of AI-driven CI/CD pipeline optimization, highlighting the growing use of machine learning

for pipeline analytics, failure prediction and automation. That work reviewed existing techniques but did not propose an implementation.

Methodology

Kaufman et al. [20] formalized *data leakage*: the use of information during training that will not be available when predicting. Bergmeir and Benítez [21] discussed how evaluation must respect time order when data are time-dependent. Nosek et al. [22] argued for pre-registration, which means fixing the analysis plan before seeing the data, to make confirmatory results credible. These works shape the evaluation protocol of this project.

Table 3.1: Literature survey summary

Ref.	Authors (Year)	Method / contribution	Limitation / gap
[11]	Hilton et al. (2016)	Usage, costs and benefits of CI in open source	Descriptive; no prediction
[12]	Beller et al. (2017)	TravisTorrent dataset of Travis CI build histories	Mixes pre-build and post-build fields; leakage risk if used naively
[4]	Rausch et al. (2017)	Empirical analysis of build failure causes (Java)	No prediction or optimization
[13]	Hassan and Wang (2017)	Change-aware build outcome prediction	No root cause analysis or deployment
[14]	Ni and Li (2017)	Cascaded classifiers for cost-effective prediction	Focus on cost; no diagnosis
[15]	Xia and Li (2017)	Comparison of ML classifiers for CI build results	No diagnosis or deployment
[18]	Gallaba et al. (2018)	Noise and heterogeneity in build data	Descriptive; no prediction
[6]	Ghaleb et al. (2019)	Causes of long CI build durations	No prediction
[19]	Durieux et al. (2019)	Analysis of 35M+ Travis CI jobs	Descriptive; no prediction
[2]	Saidani et al. (2020)	Multi-objective genetic programming for failure prediction	No root cause analysis or optimization
[5]	Durieux et al. (2020)	Restarted and flaky builds on Travis CI	Focus on flaky builds; no prediction

Ref.	Authors (Year)	Method / contribution	Limitation / gap
[16]	Chen et al. (2020)	BuildFast: history-aware outcome prediction	Does not use in-flight build context or pre-registered external tests
[17]	Jin and Servant (2020)	Cost-efficient selection of builds to run	Focus on build skipping
[3]	Barrak et al. (2021)	Random Forest replication of why builds fail	Weak on imbalanced data; Java only
[1]	Zhang et al. (2022)	BuildSheriff: test failure triage by stack-trace similarity	Java/Travis only; struggles with flaky tests
[7]	Saidani et al. (2022)	LSTM/CNN deep learning for failure prediction	High compute; limited explainability
[10]	Hong et al. (2024)	Practitioners' perceptions of failure prediction at Atlassian	Qualitative; no new model
[8]	Farihane (2025)	Mapping study of AI for CI/CD optimization	Review only; no implementation
[9]	Kumar et al. (2025)	SVM and other ML models for failure prediction	No root cause analysis or bottleneck detection

3.3 Research Gap

CI/CD tools prioritize automating and monitoring executions rather than intelligent analytics and optimization. Several papers apply machine learning to detect pipeline failures or analyze logs, but most cover only one part of the problem: some only predict failures, while others only cluster logs or detect anomalies. Very few tools handle prediction, anomaly detection, root cause analysis and optimization together. Existing models also suffer from imbalanced datasets, scalability problems and limited explainability, and most apply only to one CI/CD tool, such as Travis CI or Jenkins.

During the project, a further and more fundamental gap became clear: **how results are evaluated**.

- **Leakage.** Datasets such as TravisTorrent contain fields computed from the build log (test counts, durations) that encode the outcome. Models that use them look accurate but cannot work before a build runs.
- **Random splits.** Shuffling builds lets a model learn a repository's *future* failure rate and use it to predict the past.

- **History that ignores finish times.** A build's "previous outcome" is only known once that build has finished; using it earlier leaks information.
- **Pooled metrics.** Repositories differ greatly in failure rate, so a model can score highly overall just by recognizing risky repositories. That says little about ranking a single team's builds, which is what a team actually needs.
- **No confirmatory tests.** Results are rarely re-tested on new data under a plan fixed in advance.

There is therefore a need for an AI-based CI/CD optimization solution that combines prediction, anomaly detection, root cause analysis and visualization, and that is evaluated under a protocol matching real use. This project addresses both needs.

CHAPTER 4 – SYSTEM REQUIREMENT SPECIFICATION

4.1 Functional Requirements

The artificial intelligence-based pipeline optimization system should satisfy the following functional requirements. Section 7.8 maps each one to its implementation.

- Collection of historical CI/CD pipeline records, including build outcomes, commits, timing and logs, and live ingestion of GitHub Actions events.
- Preprocessing and cleansing of pipeline data for effective analysis.
- Feature extraction from historical records, including code churn, failure rates, commit timing and pipeline history, using only information available before a build runs.
- Training and execution of machine learning algorithms for build failure prediction.
- Anomaly detection for slow or abnormal pipeline jobs.
- Categorization of similar failure logs for automated root cause analysis.
- Optimization suggestions to improve the efficiency of pipelines.
- Visualization of pipeline analytics, predictions and bottlenecks through a dashboard.

4.2 Non-functional Requirements

- **Scalability:** The system should cope efficiently with large volumes of CI/CD data from many projects and repositories. Development used 3.56 million Travis CI job records.
- **Reliability:** The platform should produce consistent predictions. The live history computed by the service must equal the history used in training.
- **Performance:** The system should score a build quickly enough to be useful before it runs.
- **Accuracy:** Models should rank risky builds well within each repository and be evaluated without information leakage.
- **Explainability:** Each prediction should come with an explanation of the factors behind it.
- **Security:** Webhook endpoints must verify GitHub's HMAC signature, and the public demonstration must be read-only.

- **Usability:** The monitoring dashboard should be user-friendly and intuitive.
- **Maintainability:** The architecture should make maintenance and enhancement easy; automated tests and continuous integration protect it.

4.3 Hardware Requirements

Table 4.1: Hardware requirements

Component	**Specification**
Processor	Intel i5 / Ryzen 5 / Apple silicon or higher
RAM	Minimum 8 GB (16 GB recommended for training on the full Travis CI dataset)
Storage	256 GB SSD
GPU (Optional)	Not required: all models are tree ensembles trained on CPU
Operating System	Windows / Linux / macOS
Deployment	Cloud container service with 512 MB RAM (Render free tier is sufficient for the demonstration)

4.4 Software Requirements

Table 4.2: Software requirements

Software Component	**Purpose**
Python 3.11	Core programming language
Pandas, NumPy, PyArrow	Data preprocessing, analysis and storage
Scikit-learn	Random Forest, Decision Tree, SVM, Isolation Forest, TF-IDF and k-means
LightGBM, XGBoost	Gradient-boosted models, including the learning-to-rank model
SHAP	Explanations of individual predictions
FastAPI, Uvicorn	REST API and webhook service
SQLAlchemy, SQLite	Storage of pipelines, builds, predictions and job telemetry
ReportLab	Per-build PDF reports

Software Component	**Purpose**
React, Vite, Tailwind CSS	Web dashboard
Recharts	Animated charts on the dashboard
pytest	Automated tests (84 tests)
GitHub, GitHub Actions	Version control, continuous integration of this project, and a data source
Docker, Render	Containerized deployment of the public demonstration
Visual Studio Code	Development environment

CHAPTER 5 – SYSTEM DESIGN

5.1 System Architecture

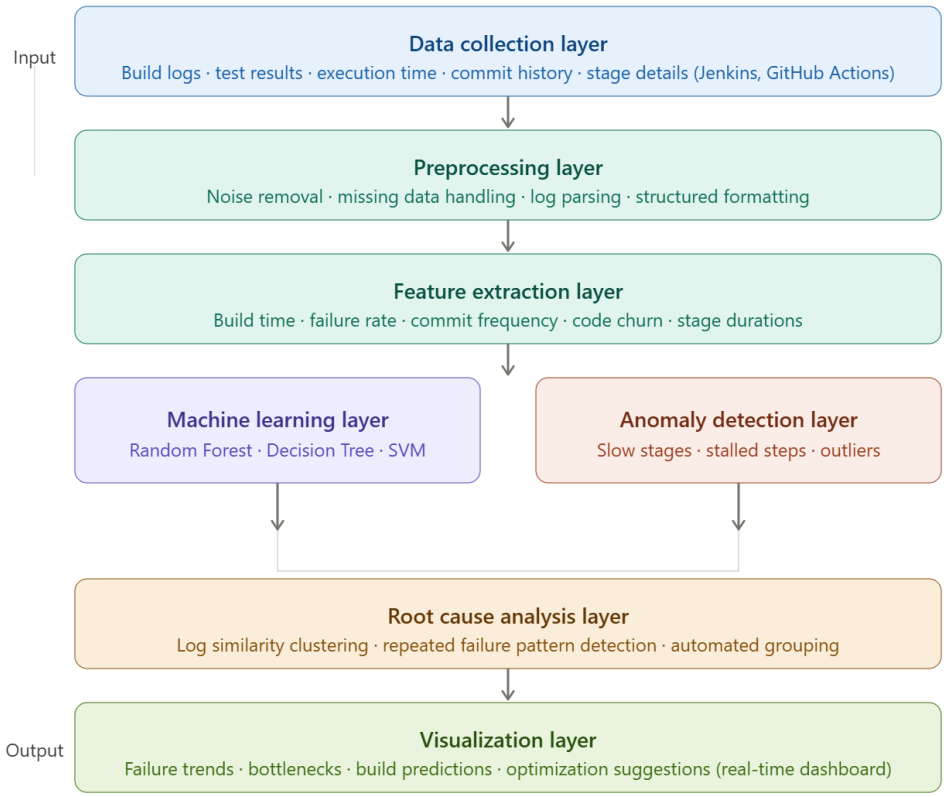


Fig 2: System architecture of the proposed CI/CD optimization framework

The framework uses a modular architecture that analyzes and optimizes CI/CD pipelines through several processing layers (Fig 2). Chapter 7 describes how each layer was implemented.

- **Data Collection Layer:** Collects historical CI/CD data, including build outcomes, logs, test results, execution times, commit history and job details. The sources are the TravisTorrent export of Travis CI and the GitHub Actions API. Live data arrives through signed GitHub Actions webhooks, and a Jenkins notification endpoint is also provided.
- **Preprocessing Layer:** Cleans and structures the data. It collapses Travis CI jobs into builds, removes duplicates and re-run attempts, sanitizes logs, and, crucially, enforces when each piece of information becomes available.
- **Feature Extraction Layer:** Extracts the size of each change, timing, and the failure history of the repository and of the build's own branch or pull request (its *stream*), using only

builds that had finished before the current build started.

- **Machine Learning Layer:** Random Forest, Decision Tree and Support Vector Machine were trained as planned, alongside XGBoost, LightGBM and a stacking ensemble. The final model, Tier E, blends a LightGBM classifier with a LightGBM learning-to-rank model.
- **Anomaly Detection Layer:** Detects jobs that run abnormally long compared with their own history, and unusual input profiles (Isolation Forest).
- **Root Cause Analysis Layer:** Groups similar failure logs (TF-IDF and k-means) and diagnoses failed-job logs with ordered rules, mapping each failure type to a suggestion.
- **Visualization Layer:** Displays predictions, alerts, failure trends, bottlenecks, explanations and suggestions on a React dashboard.

5.2 Proposed Methodology

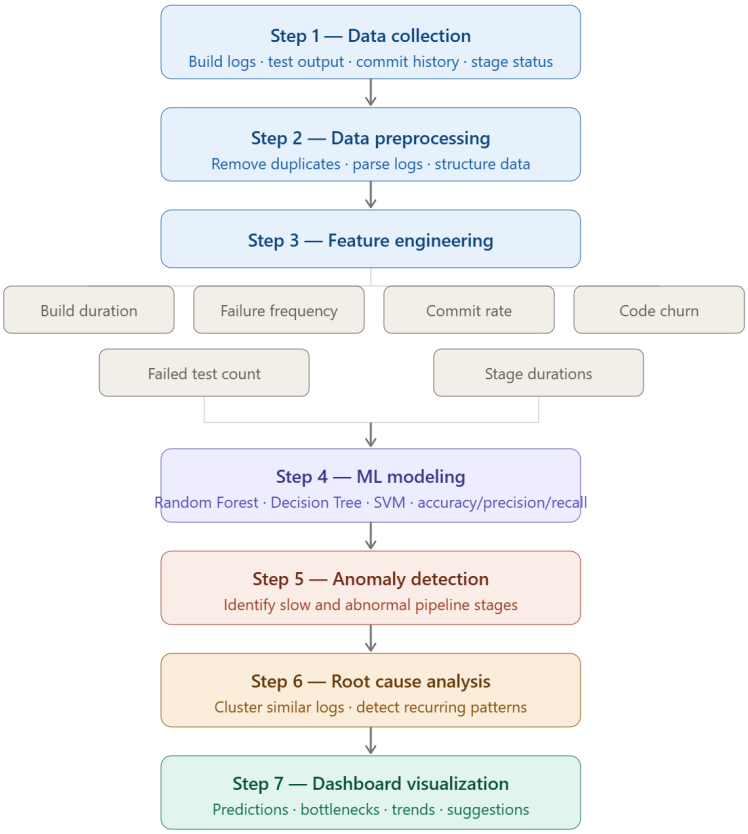


Fig 3: Proposed methodology for AI-driven CI/CD pipeline optimization

The solution follows a step-by-step methodology (Fig 3): data collection, preprocessing, feature extraction, training of predictive models, bottleneck detection, root cause analysis, and visual

dashboard recommendations.

Step 1: Data Collection

Historical CI/CD data is obtained from TravisTorrent (858,368 Travis CI builds) and from the GitHub Actions API: workflow runs, their commits, and job timings for 60 public repositories. Live GitHub Actions events arrive through webhooks.

Step 2: Data Preprocessing

The data is converted to a structured form: jobs are collapsed into builds, duplicates and re-run attempts are removed, unknown completion times are handled, and logs are sanitized. Every input is checked against an *availability registry* that records whether it is known before the build starts.

Step 3: Feature Engineering

Features that describe the change and its context are extracted from the cleaned data:

→ size of the code change (lines and files changed, by type)

→ failure frequency of the repository, and of the build's own branch or pull request

→ outcome of the previous build, and whether it is still running

→ commit timing and time of day or week

→ number of builds running in the pipeline and started in the last hour

→ job durations, used for bottleneck detection after a build runs

Build duration and failed test counts are known only after a build finishes. They are deliberately *not* used to predict failures, because doing so leaks the outcome (Section 8.2).

Step 4: Machine Learning Modeling

Random Forest, Decision Tree, SVM and gradient-boosted models classify whether a build will fail. They are assessed with accuracy, precision, recall and F1, as planned. The primary metric is within-repository AUC under chronological walk-forward evaluation, because it matches how a team would use the predictions.

Step 5: Anomaly Detection

Slow or stalled jobs are detected by comparing each job's duration with the history of the same job, and unusual input profiles are flagged with Isolation Forest.

Step 6: Root Cause Analysis

Failure logs are grouped by textual similarity and diagnosed with ordered rules to detect recurring failure patterns. This reduces debugging effort.

Step 7: Dashboard Visualization

Predictions, alerts, failure trends, bottlenecks and recommendations are presented on an interactive dashboard to help developers analyze pipeline behavior.

5.3 Detailed Design

5.3.1 Class Diagram

The detailed design describes the inner workings of, and interactions between, the system's components. Fig 4 shows the main classes as implemented.

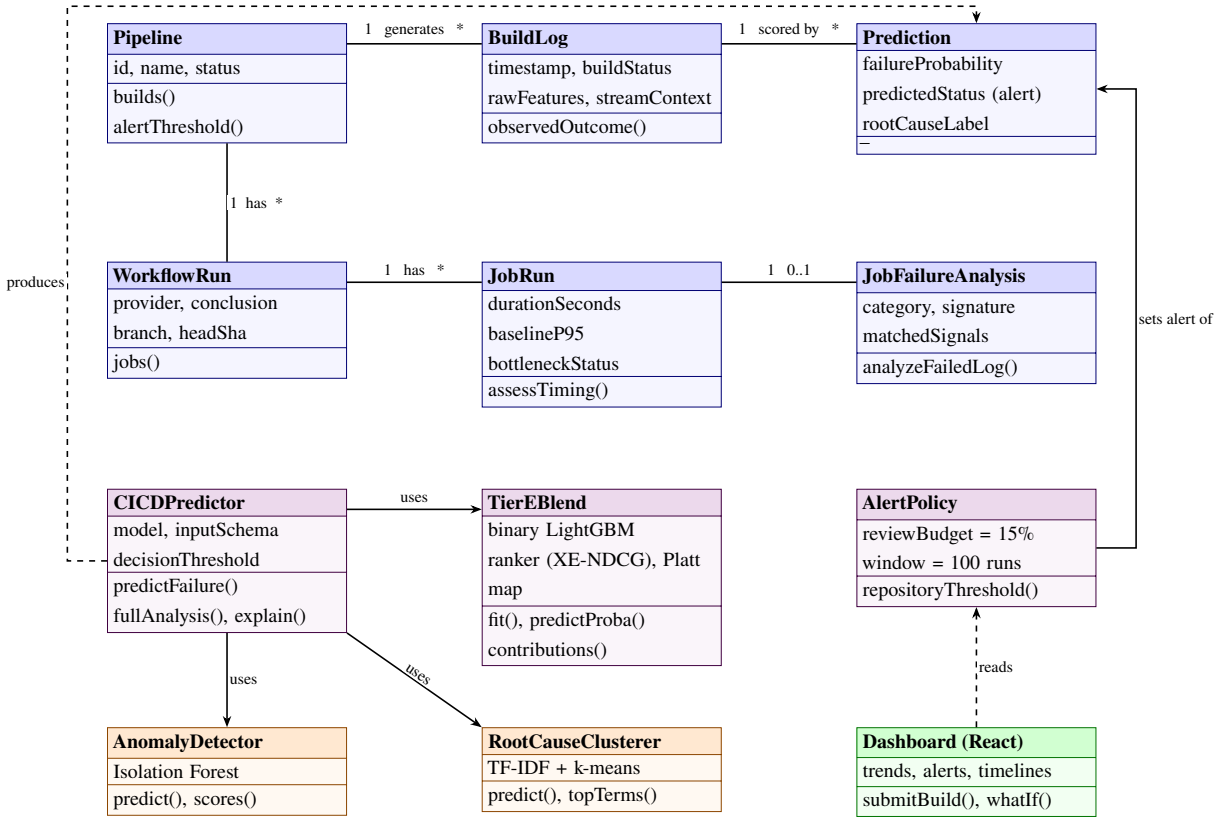


Fig 4: Class diagram of the CI/CD optimization system

- **Data classes.** `Pipeline` represents a repository. A `BuildLog` stores one build or workflow run, its observed outcome and the stream history derived for it, and each `Prediction` stores its risk, alert decision and root cause. `WorkflowRun` and `JobRun` store job telemetry used for bottleneck detection, and `JobFailureAnalysis` stores the diagnosis of a failed job's log.

- **Prediction classes.** `CICDPredictor` loads the deployed model and scores builds using

`TierEBlend`. `AlertPolicy` turns a score into an alert using the repository's own review budget.

- **Analysis classes.** `AnomalyDetector` and `RootCauseClusterer` provide anomaly detection and log clustering.
- **Dashboard.** The React dashboard presents the results to the user.

The design originally included a `User` class with login. Sign-in was later removed: the dashboard is an internal or read-only tool, and machine endpoints are protected by webhook signatures and shared secrets instead.

5.3.2 Activity Diagram

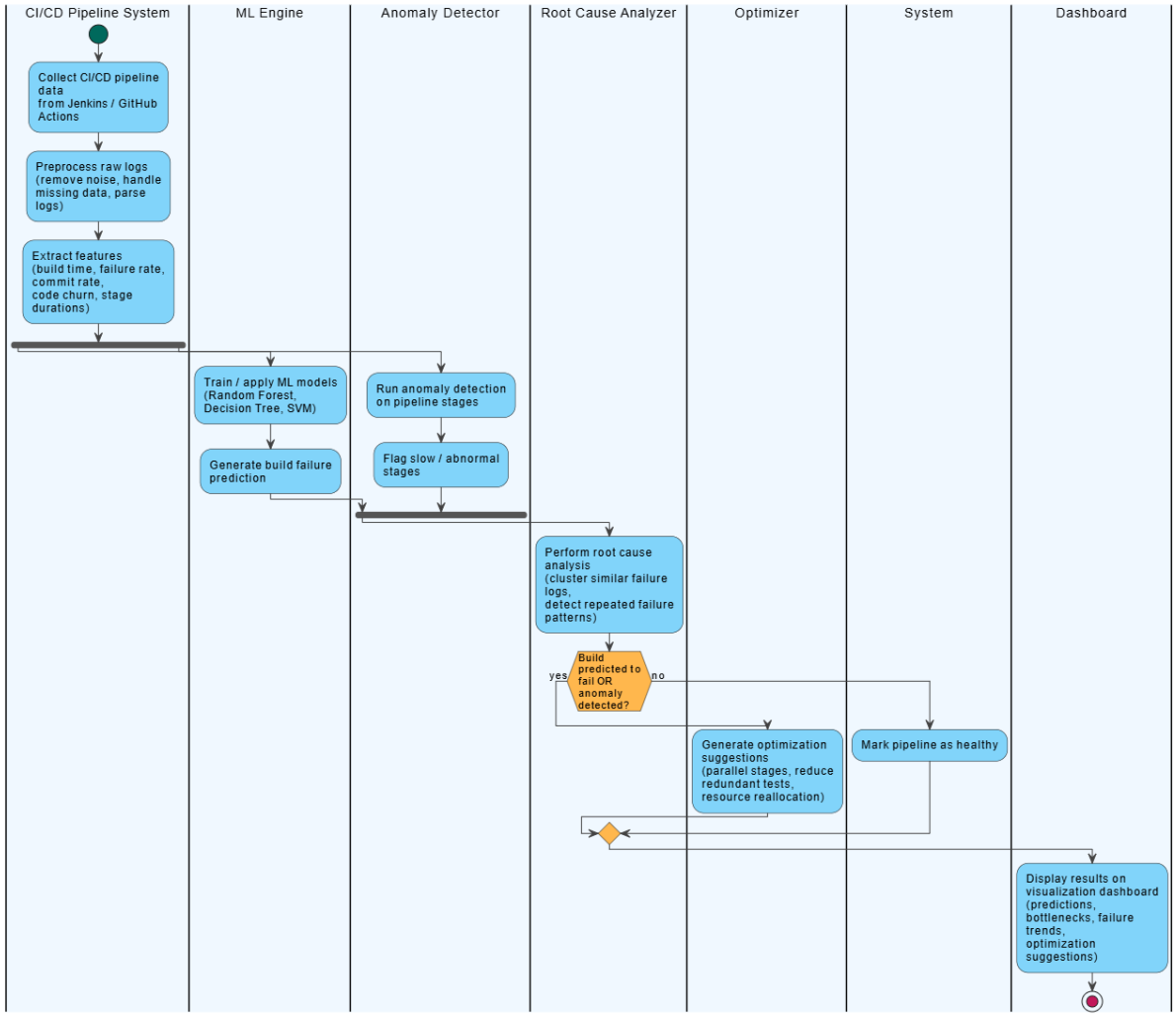


Fig 5: Activity diagram of the CI/CD optimization system

The activity diagram depicts the flow of events in the system (Fig 5).

1. CI/CD pipeline data is gathered from repositories and build systems.

2. The data is preprocessed and passed to feature engineering.

3. Machine learning models produce build failure predictions. In the deployed system this happens when a GitHub Actions run is requested, before it executes.

4. The anomaly detection and root cause analysis components detect bottlenecks and recurring failure patterns.

5. When a build is predicted to fail or an anomaly is found, optimization suggestions are generated.

6. Finally, the results are displayed on the dashboard.

5.3.3 Use Case Diagram

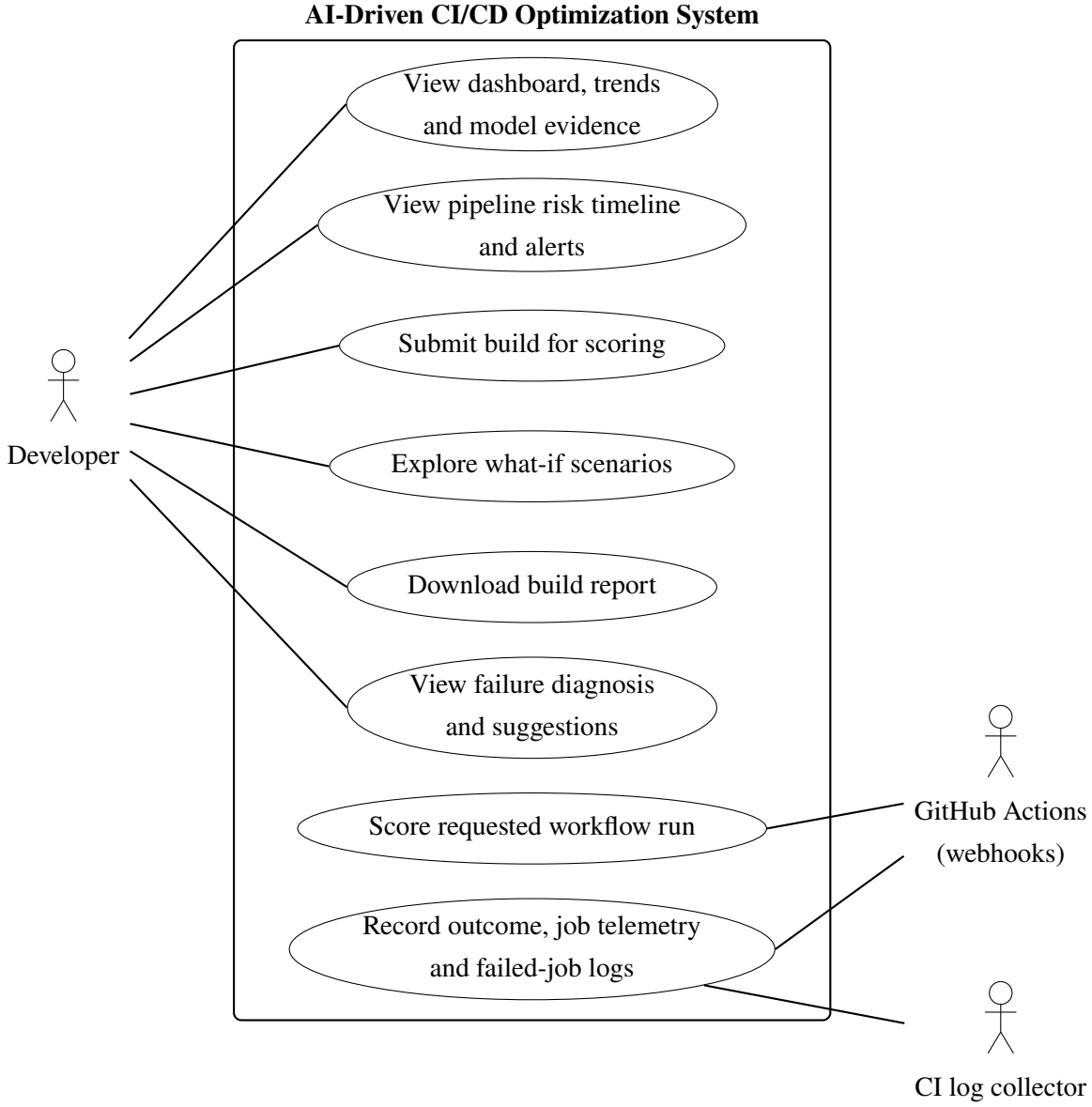


Fig 6: Use case diagram of the CI/CD optimization system

The use case diagram (Fig 6) shows how users and external systems interact with the solution.

Actors

- **Developer:** views the dashboard, pipelines and alerts, scores builds and explores what-if scenarios.
- **GitHub Actions:** sends signed workflow-run and job events.
- **CI log collector:** an optional workflow that uploads failed-job logs.

Use Cases

- View dashboard, failure trends and model evidence
- View a pipeline's risk timeline and alerts
- Submit a build for scoring
- Explore what-if scenarios
- Download a per-build PDF report
- View failure diagnosis and optimization suggestions
- Score a requested workflow run automatically
- Record run outcomes, job telemetry and failed-job logs

CHAPTER 6 – IDENTIFICATION OF TOOLS AND TECHNIQUES

6.1 Development Tools

The system uses tools chosen for scalability, compatibility, and their support for machine learning, data analysis and pipeline integration.

Python 3.11 is the primary programming language, chosen for its simplicity, cross-platform support and extensive ecosystem of machine learning and data processing libraries. All package versions are pinned so that saved model artifacts load exactly as trained.

Scikit-learn [32] implements Random Forest, Decision Tree and Support Vector Machine for build failure prediction, along with Isolation Forest, TF-IDF and k-means for anomaly detection and log clustering, and the evaluation metrics.

LightGBM [27] and **XGBoost** [26] provide gradient-boosted trees. LightGBM also provides the learning-to-rank objective used by the final model.

Pandas, NumPy and PyArrow are used for data preprocessing, cleaning, feature extraction, numerical computation and efficient storage of the 3.56 million-row Travis CI data.

SHAP [30] explains individual predictions by attributing each prediction to its input features.

Visual Studio Code is the main development environment, and **pytest** runs the automated test suite.

Deep learning frameworks (TensorFlow, PyTorch) were planned for possible future use. They were not needed: tree ensembles performed well, are faster to train, and are easier to explain.

6.2 CI/CD and Integration Tools

GitHub Actions is a cloud-based CI/CD platform tightly coupled with GitHub repositories. In this project it plays three roles:

- **Data source:** workflow runs, commits and job timings of 60 public repositories were collected through its API.
- **Live input:** the deployed system receives signed `workflow_run` and `workflow_job` webhook events and scores each run when it is requested.
- **CI for this project:** every pull request runs the backend tests and a production build of the dashboard.

Travis CI, through the TravisTorrent dataset [12], provides the main development data: 3.56 million job records from 947 projects.

Jenkins is one of the most widely used open-source CI/CD automation servers. The system provides a Jenkins notification endpoint protected by a shared secret, so Jenkins builds can be recorded and scored as well.

GitHub hosts the project's code and is used for version control, pull requests and releases. Commit activity on GitHub, such as code churn, files changed and commit timing, also provides important model features.

6.3 Visualization and Deployment Tools

The design originally proposed Grafana and Kibana for visualization. Instead, a purpose-built **React** dashboard was implemented with **Vite**, **Tailwind CSS** and **Recharts**. It shows exactly the project's outputs: failure risk, alerts, trends, explanations, bottlenecks and model evidence. It uses animated, colorblind-checked charts, and every chart has a legend or labels. The backend is a **FastAPI** service with a **SQLite** database (via SQLAlchemy). **ReportLab** generates per-build PDF reports.

Docker packages the service. A read-only public demonstration runs as a single container on **Render**, serving the dashboard and the API from one address.

6.4 Techniques Used

Random Forest

Random Forest [23] is an ensemble of decision trees trained on random subsets of data and features. It is robust against overfitting and handles mixed, complex features well. In this project it is the main baseline: Tier C, a Random Forest with repository history, is the model that Tier E is compared against.

Decision Tree

A Decision Tree [24] classifies data with a hierarchy of decision rules derived from the features. It is interpretable, which helps explain what drives pipeline failures. It was trained and compared as planned.

Support Vector Machine (SVM)

SVM [25] finds the boundary that best separates classes in a high-dimensional feature space. It was evaluated against Random Forest and Decision Tree using accuracy, precision, recall and F1-score.

Gradient Boosting and Learning to Rank

Gradient boosting (XGBoost [26], LightGBM [27]) builds trees sequentially, each correcting the errors of the previous ones. A team's real question is "which of *our* builds are riskiest?", so the final model also uses a *learning-to-rank* objective, XE-NDCG [28]. It learns to order builds within each repository and month rather than to classify them in isolation. Platt scaling [29] converts the blended score into a probability.

SHAP Explanations

SHAP values [30] attribute each prediction to its input features, such as "the previous run of this workflow failed" or "many lines changed". The dashboard shows them as red (raises risk) and blue (lowers risk) bars.

Anomaly and Bottleneck Detection

Bottleneck detection compares a job's duration with at least 10 earlier runs of the same job. A job is flagged when it exceeds both their 95th percentile and a robust threshold of the median plus three median absolute deviations. Isolation Forest [31] flags unusual pre-build input profiles. It is not a failure predictor: in testing, anomalous builds did not fail more often, so the dashboard does not present it as one.

Log Clustering and Diagnosis

Failure logs are represented with TF-IDF term weights [33] and grouped with k-means clustering [34], so that similar failures fall in the same group (Fig 7). Failed-job logs from GitHub Actions are also sanitized and matched against ordered diagnostic rules, which report the matched lines and suggest a fix. The categories are resource or runner exhaustion, timeouts and network instability, dependency or build-configuration failures, compilation errors, and test assertion failures.

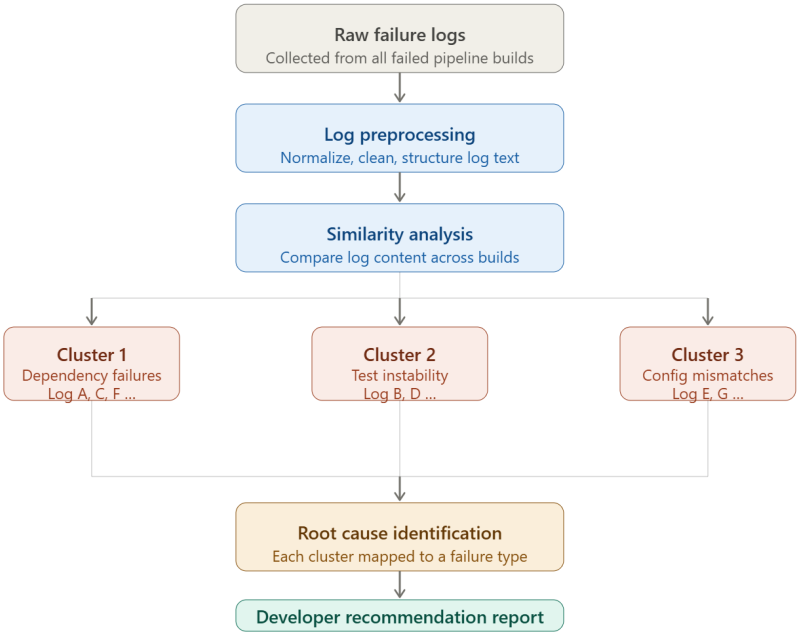


Fig 7: Log clustering flow for root cause analysis

Feature Engineering

Meaningful attributes are extracted from raw CI/CD data: code churn, files changed by type, failure rates of the repository and of the build's branch or pull request, the outcome of the previous build, builds still running, and commit timing. Every feature is checked against the availability registry, so nothing that is only known after a build finishes can be used (Section 7.2).

Leakage-Safe Evaluation

Models are evaluated with chronological walk-forward folds: they are trained only on the past and tested on the future. The metric is AUC measured within each repository, and confidence intervals come from bootstrapping over repositories [35]. External tests were pre-registered [22]: their plan was fixed and fingerprinted before any data was collected.

CHAPTER 7 – IMPLEMENTATION

7.1 Datasets

Three datasets were used (Table 7.1).

Travis CI (development)

The TravisTorrent export [12] contains 3.56 million job records. A Travis build runs several jobs, for example on different language versions. We collapsed the jobs into **858,368 builds** from **947 projects** (April 2011 to August 2016), checking that every pre-build field is identical across a build's jobs. A build fails if any of its jobs failed; 19.5% of builds failed. A build's finish time is its start time plus its longest job's duration. Missing durations are filled with the repository's 95th-percentile duration.

GitHub Actions (external tests)

On GitHub Actions the unit is a *workflow run*. We kept first attempts of `push` and `pull_request` runs. `success` is a pass, and `failure` and `startup_failure` are failures; cancelled and skipped runs are excluded. A run's finish time is its `updated_at`, and runs whose completion time is unknown are dropped. The size of each change (lines added and deleted, files changed, message length, parents) comes from the commits API.

The *first* GitHub Actions dataset covers 30 repositories. The *confirmation* dataset was collected with our own collector under a protocol fixed in advance. It takes the first 30 eligible repositories from a search of popular repositories, excluding all repositories used before, with runs created between 15 August and 30 September 2026. Of the qualifying runs, 3,773 (4.6%) were dropped because their commits, mostly on deleted forks, could no longer be fetched.

Table 7.1: Datasets used in the project

Dataset	Builds / runs	Repos	Failure rate	Period
Travis CI (TravisTorrent)	858,368	947	19.5%	2011–2016
GitHub Actions, test 1	67,980	30	8.9%	2025–2026
GitHub Actions, confirmation	78,576	30	28.2%	Aug–Sep 2026
GitHub Actions, combined	146,556	60	19.2%	2025–2026

7.2 Leakage-Safe Data Pipeline

Most of the effort went into making sure that no model ever sees information it would not have when a build is triggered.

Availability registry

Every column is recorded in a versioned registry with the stage at which it becomes known: pre-build, build start, or post-build. Fields computed from the build log, such as test counts, test duration and build duration, are marked post-build. So are comment counts, which can be written after the build. The predictor checks its inputs against the registry when it starts and refuses to run if any input is not available at prediction time.

Finish-time-safe history

History features count a previous build only if it *finished* strictly before the current build was triggered. Builds are processed per repository in time order, with a queue of builds still running; a build's outcome enters the history only once its finish time has passed. Ties are broken by finish time, then trigger time, then build id. In every evaluation run, a brute-force recomputation re-checks the history values on sampled repositories, and a deliberately leaked feature makes this check fail.

Finish buffers

To test robustness, outcomes can be withheld for an extra 1 or 6 hours after a build finishes, which simulates delays in reporting.

7.3 Feature Engineering

Features are organized in tiers (Table 7.2). A *stream* is the build's pull request or, for push builds, its branch. On GitHub Actions, runs carry no pull request number, so a pull request is identified by its author and head branch.

Table 7.2: Feature tiers

Tier	Features
Tier A	**The change and its context** (18 features): source and test churn; files added, deleted and modified by type; hour, day, month, weekend and night flags; pull request flag; core-team author; default branch; and the project language. On GitHub Actions: lines added and deleted, files changed, message length, merge commit, default branch, and the workflow.

Tier	Features
Tier C	Tier A plus **project context** (team size, code size, test density, past commits on the touched files, commits in the build) and **repository history**: the previous finished build's outcome and the repository's failure rate over the last 5 builds and overall.
Tier E	Tier C plus **stream history**: the outcome of the stream's last finished build; the stream's last-5 and overall failure rates, finished-build count and failure streak; the outcome of the build's predecessor (on Travis, the previous build on the branch; on GitHub Actions, the previous run of the same workflow on the stream); and an exponentially weighted repository failure rate. It also adds **in-flight context**: whether the previous build and the stream's last build are still running, how many builds are running in the repository, how many started in the last hour, and the hours since the stream last triggered. In-flight features use only the existence and start times of earlier builds, never their outcomes.

7.4 Prediction Models

Planned classifiers

As planned, Random Forest, Decision Tree and SVM were trained, alongside XGBoost, LightGBM and a stacking ensemble (Random Forest, XGBoost and LightGBM combined by logistic regression). Hyperparameters were tuned with randomized search (25 settings, 3-fold cross-validation, F1 score) on a 200,000-row sample. The comparison is reported in Section 8.2 as a *development reference only*, because it uses a random split.

Final model: Tier E

The final model combines two LightGBM models trained on Tier E features:

- a binary classifier, which predicts whether a build fails;
- a ranker trained with the XE-NDCG objective [28], which learns to order builds within one repository and calendar month (one ranking "query").

Let $b(x)$ be the classifier's log-odds and $r(x)$ the ranker's score. Both are standardized with the mean and standard deviation of the *training* scores and averaged:

$$s(x) = \tfrac{1}{2}\,\frac{r(x) - \mu_r}{\sigma_r} + \tfrac{1}{2}\,\frac{b(x) - \mu_b}{\sigma_b}. \tag{7.1}$$

A Platt map [29] fitted on training scores turns $s(x)$ into a probability. LightGBM settings were fixed before any confirmation data was scored: 600 trees, learning rate 0.03, 63 leaves, at least 200 samples per leaf, 80% row and column sampling, and L2 regularization 1.0.

Deployed model

The deployed model is trained on walk-forward folds 1–9 of the 146,556 GitHub Actions runs, with the newest 10% held out. Before it is saved, it must reproduce the research evaluation's predictions on the held-out fold; the largest difference was 0. Explanations use SHAP values of the binary half of the model.

7.5 Anomaly, Bottleneck and Root Cause Modules

- **Bottleneck detection.** For every completed GitHub Actions job, the system compares its duration with at least 10 earlier runs of the same job. A job that exceeds both their 95th percentile and the median plus three median absolute deviations is marked as a bottleneck. A scheduled collector also checks jobs that are still running, so stalled jobs are flagged while they run.

- **Anomaly detection.** An Isolation Forest (10% contamination) flags builds whose pre-build input profile is unusual. Because such builds did not fail more often, the flag is kept as a diagnostic and not shown as a failure prediction.

- **Root cause analysis.** Failed-test text is clustered with TF-IDF and k-means. Failed GitHub Actions job logs, uploaded by an optional workflow, are sanitized and diagnosed with ordered rules into five categories. Each category maps to a suggestion, for example retrying transient tests and mocking external calls, or pinning compatible dependency versions.

7.6 Deployed System

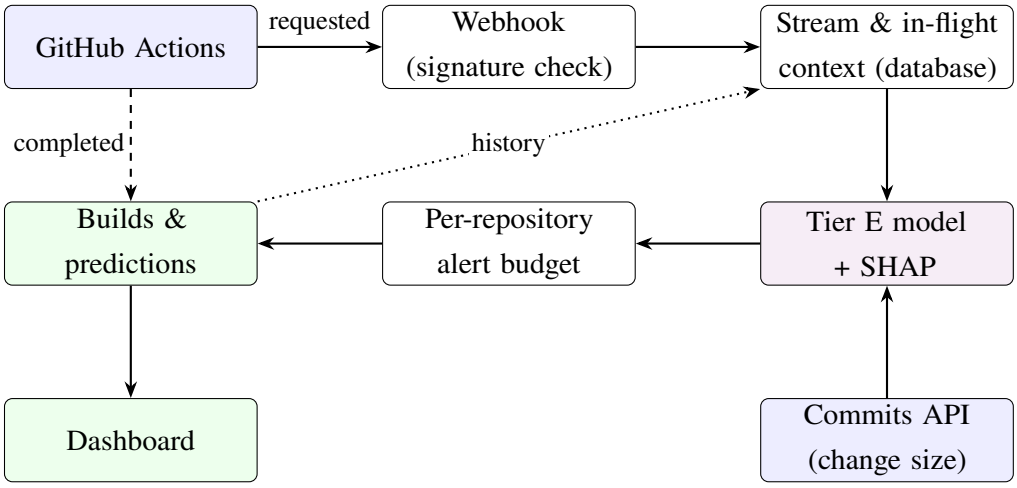


Fig 8: Scoring path of the deployed system

Scoring path

Fig 8 shows the scoring path. A FastAPI service receives signed GitHub `workflow_run` events.

1. When a run is *requested*, before it executes, the service builds the run's stream history and in-flight context from the repository's stored runs, and fetches the commit's size from the GitHub API.

2. It scores the run with Tier E and stores a shadow prediction with its explanation.

3. When the run *completes*, its real outcome is attached and becomes history for later runs. Other conclusions, such as cancelled runs, are stored but never become history, as in training.

Runs are timed by GitHub's own creation and completion times, so a late webhook delivery cannot change a run's history. The system is advisory and never blocks CI.

Same history live as in training

Automated tests replay synthetic run histories through both the offline training pass and the live service, and check that every history and in-flight value matches, row by row.

Per-repository alert budget

A single global threshold flags too many runs in risky repositories and almost none in safe ones. Instead, a run becomes an alert when its risk reaches the 85th percentile of its *own repository's* last 100 scored runs: each repository reviews its riskiest 15%. Until a repository has 20 scored runs, a "typical repository" threshold of 0.893 is used. Neither threshold uses outcome labels. Section 8.6 evaluates this policy.

Public demonstration

A read-only demonstration runs on Render as a single Docker container that serves the dashboard and the API from one address. It replays 450 real GitHub Actions runs from `twbs/bootstrap`, `freeCodeCamp` and `yt-dlp` through the real webhook, at their real times. The runs come from the model's held-out period, so every prediction shown is out-of-sample, and contributor and branch names are replaced with pseudonyms. Visitors can explore every page and score builds, but nothing is saved.

7.7 Dashboard

The dashboard has four pages.

Dashboard

Summary cards, the daily trend of average predicted risk against the actual failure rate (Fig 9), the distribution of predicted risk, and a model-evidence panel with the research results (Fig 21, Chapter 8).

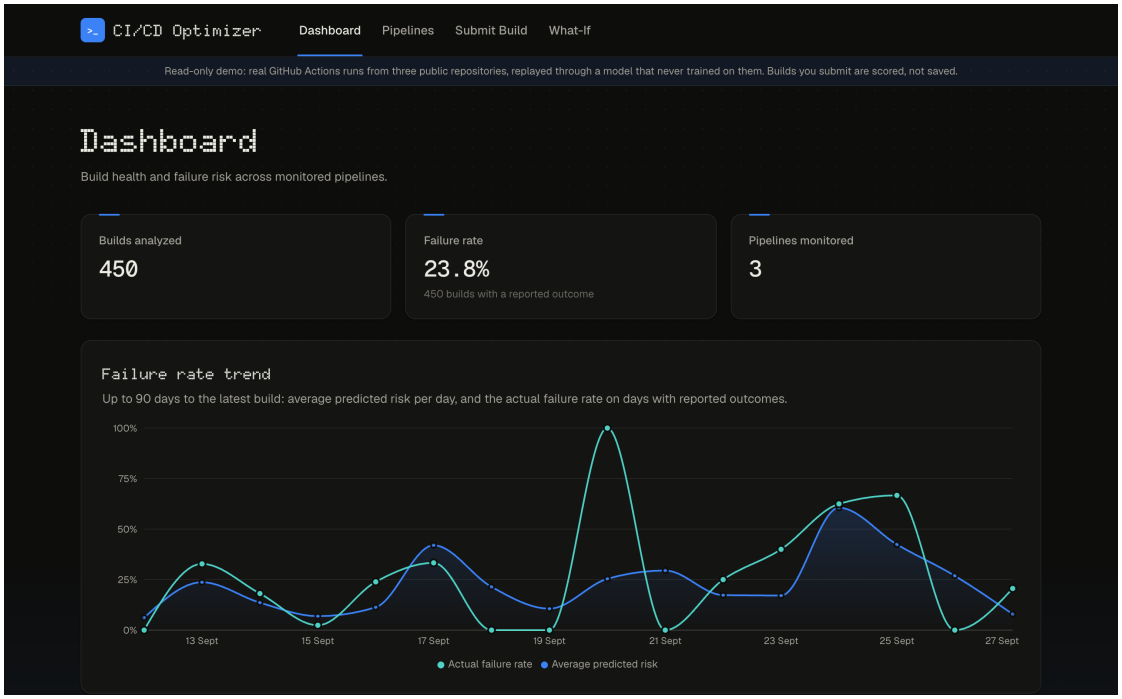


Fig 9: Dashboard: summary cards and daily failure-rate trend (public demonstration)

Pipelines

Each repository's latest status and risk, with a sparkline of its recent risk (Fig 10). A repository's page shows its risk over time, with each run colored by its real outcome and the repository's alert threshold as a dashed line (Fig 11). It also lists every build with its explanation and a downloadable PDF report.

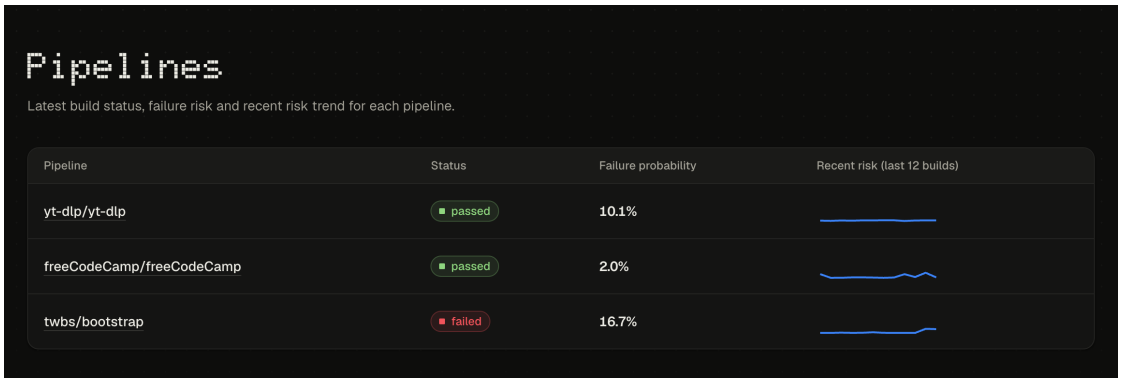


Fig 10: Pipelines page with recent-risk sparklines

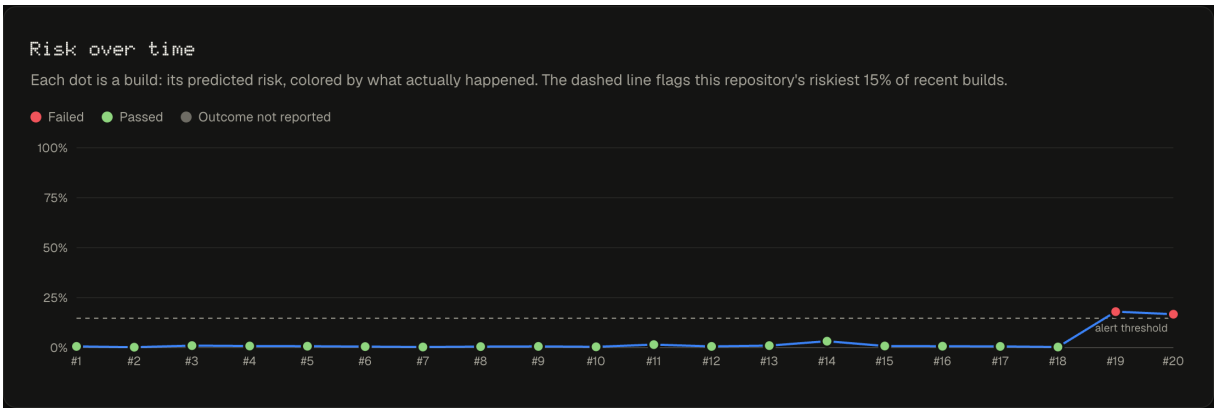


Fig 11: Risk timeline of `twbs/bootstrap`. The two newest runs, both real failures, rise above the repository's alert threshold.

Submit Build

Scores a GitHub Actions run described by its repository, workflow, pull request author and branch, and commit size. It shows the history the server derived for it and compares the risk with the repository's own threshold (Fig 12).

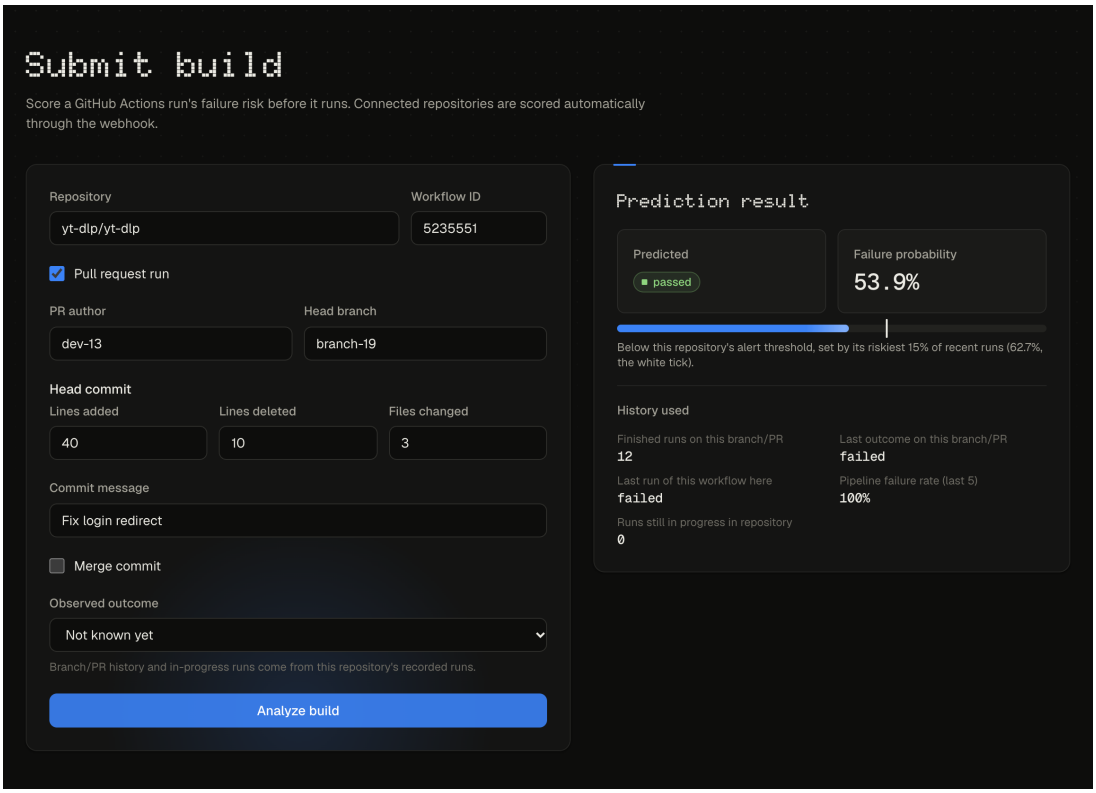


Fig 12: Submit Build: a run on a failing `yt-dlp` branch, scored with server-derived history

What-If

Changes inputs and immediately shows how the risk moves, with SHAP explanations and a trail

of the risk across edits (Fig 13). An optimization simulation compares one changed input against the baseline.

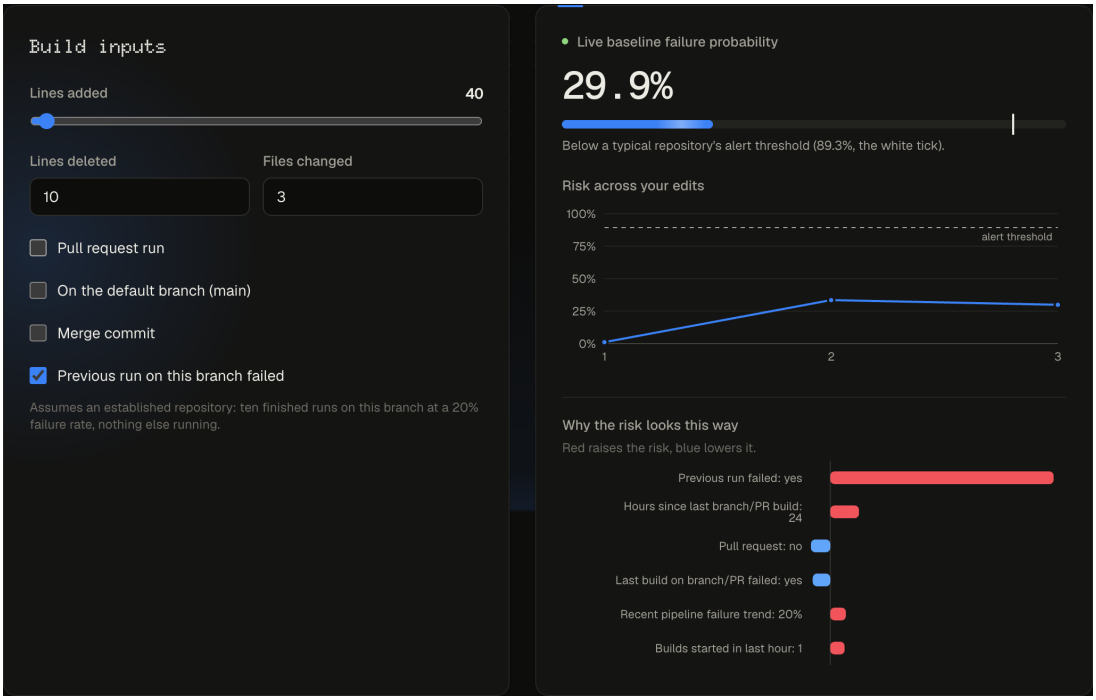


Fig 13: What-If page: whether the previous run of the same workflow failed dominates the explanation

7.8 Testing and Requirement Traceability

The project has 84 automated tests, run by GitHub Actions on every pull request together with a production build of the dashboard. They cover:

- feature engineering, the availability registry and live-versus-training history equivalence;
- the deployed model and the alert policy;
- webhook signatures and the requested-to-completed flow, including late deliveries;
- database migrations, the dashboard endpoints, PDF reports and the read-only demonstration mode.

Table 7.3 maps the requirements of Chapter 4 to their implementation.

Table 7.3: Requirement-to-implementation traceability

Requirement	Implementation	Verified by
Data collection	TravisTorrent ingestion; GitHub Actions collector; signed webhooks	Webhook and signature tests

Requirement	**Implementation**	**Verified by**
Preprocessing	Build collapsing, label rules, availability registry	Preprocessing and registry tests
Feature extraction	Finish-time-safe history, stream and in-flight context	Brute-force and equivalence tests
Failure prediction	Tier E model (LightGBM blend); RF, DT, SVM baselines	Model tests; reproduction check
Anomaly / bottleneck detection	Job timing vs. history; Isolation Forest	Bottleneck and anomaly tests
Root cause analysis	TF-IDF + k-means; rule-based log diagnosis	Clustering and log-analysis tests
Optimization suggestions	Category-to-suggestion rules; what-if simulation	Suggestion and simulator tests
Dashboard	React dashboard with charts, reports and alerts	Endpoint tests; production build

CHAPTER 8 – RESULTS AND DISCUSSION

8.1 Evaluation Protocol

All headline results use the following protocol, designed to match how a team would use the predictions.

- **Walk-forward folds.** Builds are sorted by trigger time and split into 10 equal chronological folds. Fold k is tested after training on folds $1, \ldots, k-1$, and the test folds are 6–10. On Travis CI every design choice for Tier E was made on fold 6 alone; folds 7–10 were then scored once.
- **Metric.** The primary metric is *macro within-repository AUC*: the AUC computed within each repository that has both passes and failures in the test fold, averaged over repositories. 0.5 means random ranking and 1.0 perfect ranking. We also report *recall@15%*, the share of a repository's failures found when it reviews its riskiest 15% of builds.
- **Uncertainty.** Differences between models are paired by repository, and 95% confidence intervals come from 1,000 bootstrap resamples of repositories [35].
- **Controls.** When training labels are randomly shuffled, the ranker drops to 0.48–0.50 within-repository AUC, as it should, which confirms that no label leaks into the features.
- **Pre-registration.** Each GitHub Actions test followed a written protocol, fingerprinted (SHA-256) and committed before its data was collected [22]. Its decision rule was fixed in advance: *confirms* if the mean gain over folds 6–10 is positive and at least 4 of 5 fold confidence intervals exclude zero; *inconclusive* if the mean gain is positive otherwise; *does not replicate* if it is not positive.

8.2 How Much the Usual Evaluation Overstates Results

Table 8.1 and Fig 14 compare the six classifiers in the way most studies do: a random 80/20 split of a 200,000-row sample. On this split, Random Forest, XGBoost and the stacking ensemble look strong (AUC 0.90–0.91, F1 about 0.73), while Decision Tree and SVM trail.

Table 8.1: Development comparison of classifiers (random split; reference only)

Model	Accuracy	Precision	Recall	F1	AUC
Stacking (RF + XGB + LGBM)	0.899	**0.831**	0.653	**0.731**	**0.912**
Random Forest	0.889	0.749	0.714	0.731	0.908
XGBoost	0.889	0.752	0.709	0.730	0.900
LightGBM	0.877	0.699	0.729	0.714	0.898
Decision Tree	0.769	0.469	**0.741**	0.575	0.825
SVM	0.758	0.447	0.623	0.520	0.772

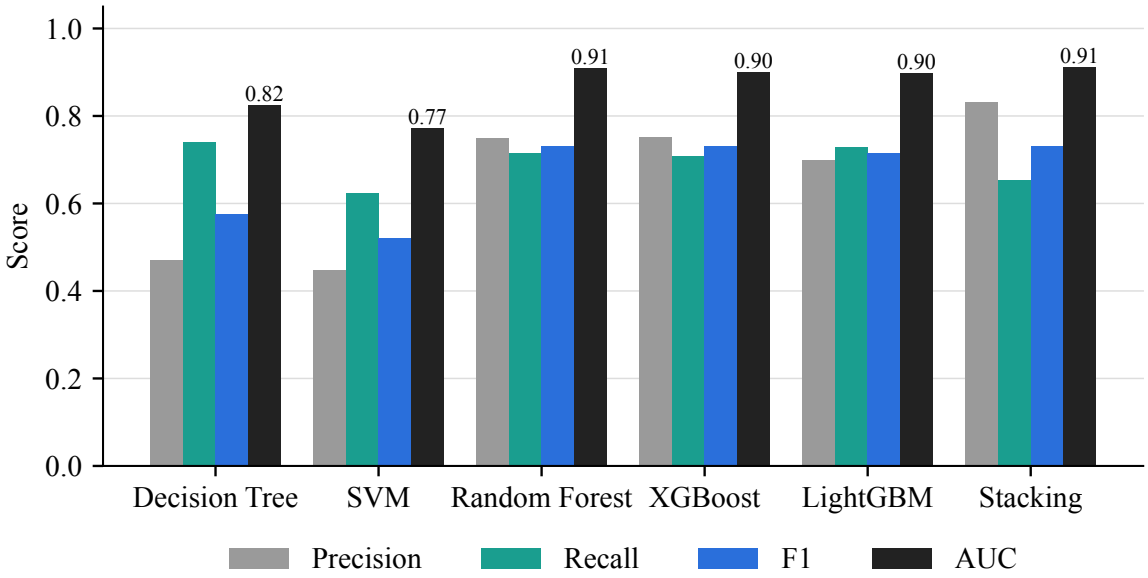


Fig 14: Development comparison of six classifiers on a random split (reference only)

These numbers do *not* describe real use. A random split lets a model learn each repository's future failure rate from builds that come later in time. Table 8.2 and Fig 15 show the same task as the evaluation becomes progressively stricter:

- On a random split, adding fields from the build log lifts AUC from 0.690 to 0.896. Those fields are not available before a build runs, and the availability registry now blocks them.
- Without leaky fields, the random-split AUC of 0.91 falls to 0.82 on a chronological split.
- It falls to **0.63** when measured within each repository at the build level, which is how a team would experience it.

The rest of this chapter reports only this strict setting.

Table 8.2: The same task under progressively stricter evaluation (Travis CI)

Setup	**AUC**
Random split, 31 features including post-build log fields (leaky)	0.896
Random split, 13 strict pre-build features	0.690
Random split, 39 features with repository history (stacking)	0.912
Chronological split, job level, pooled over repositories (Random Forest)	0.821
Walk-forward, build level, *within-repository* (Tier C Random Forest)	0.628
Walk-forward, build level, *within-repository* (Tier E, this project)	**0.698**

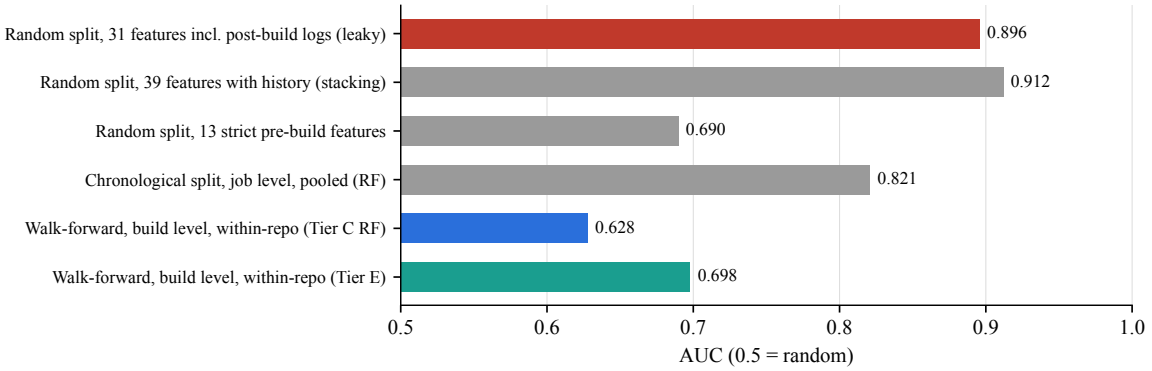


Fig 15: AUC under progressively stricter evaluation. Red: uses leaked post-build fields. Blue and teal: the strict protocol used for all results that follow.

8.3 Results on Travis CI

Fig 16 summarizes every evaluation. On Travis CI, Tier E raises macro within-repository AUC from 0.628 to **0.698** (+0.070), and every confirmation fold's 95% confidence interval is above zero. The per-fold gains are +0.068, +0.067, +0.070 and +0.074. Recall@15% within each repository improves by +0.068.

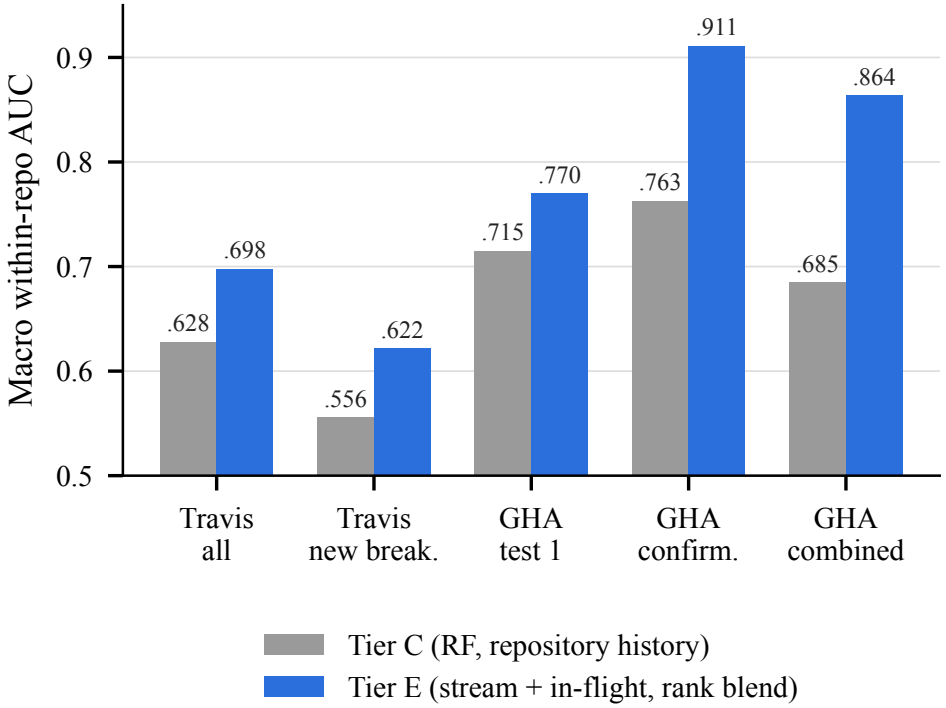


Fig 16: Macro within-repository AUC of Tier C (Random Forest with repository history) and Tier E across all evaluations (test folds; outcomes known as soon as builds finish)

Robustness to reporting delays

Table 8.3 repeats the evaluation with outcomes withheld for an extra 1 or 6 hours. Unlike a pure "it failed last time" effect, the gain *grows* as outcomes are withheld longer (Fig 17). The in-flight features, such as whether the previous build is still running, stay informative when recent outcomes are not yet known.

Table 8.3: Travis CI results by finish buffer (folds 7–10). Every fold's 95% CI for the gain is above zero.

Finish buffer	Tier C Random Forest	Tier E	Gain
0 h	0.628	**0.698**	+0.070
1 h	0.600	**0.681**	+0.081
6 h	0.585	**0.672**	+0.087

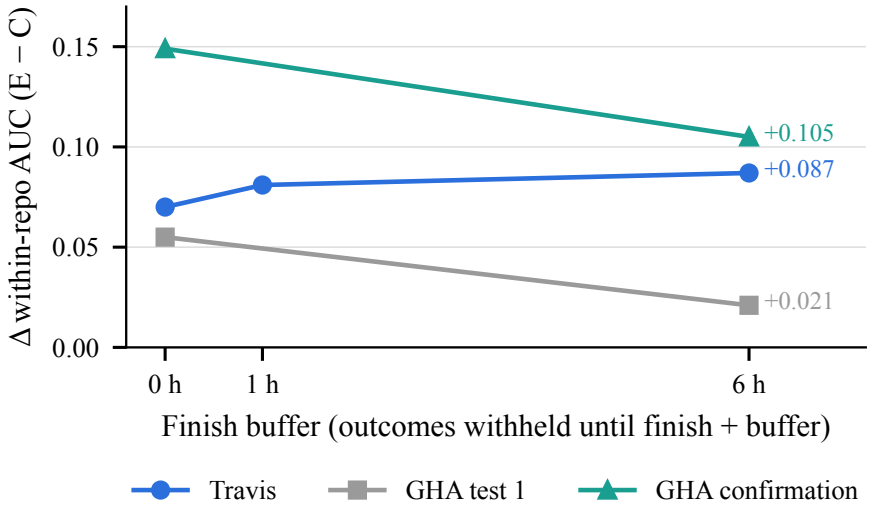


Fig 17: Gain of Tier E over Tier C as outcomes are withheld longer

What drives the gain

Table 8.4 separates the ingredients.

- The learning-to-rank objective is the largest single lever on Travis CI: it is worth +0.031 on Tier C features alone.
- Stream and in-flight features add about +0.03 more.
- Build-matrix size, the number of jobs, adds only about 0.003, so the deployed variant omits it.

The ranker trades overall (pooled) AUC for within-repository ordering. Teams that need a pooled ranking across repositories should use the binary Tier E model, which beats Tier C on both measures.

Table 8.4: Attribution of the Tier E gain on Travis CI (folds 7–10)

Variant	Within-repo AUC	Pooled AUC
Tier C, Random Forest	0.628	0.856
Tier C, binary LightGBM	0.633	–
Tier C, LightGBM ranker	0.664	–
Tier E, binary LightGBM	0.652	**0.881**
Tier E, LightGBM ranker	0.696	–
Tier E, blend	**0.698**	0.828
Tier E, blend without job counts	0.695	0.826

8.4 New Breakages

Much of the signal comes from branches or pull requests that are already failing: 55% of failures occur on streams whose last known outcome was a failure. To check that the model does more than repeat the last outcome, the saved predictions were rescored on *new breakages*, builds whose previous build on the same stream passed. These are 25% of failures, and such builds fail only 6.6% of the time.

- On new breakages, the repository's recent failure rate is at chance (0.504) and Tier C scores 0.556.
- Tier E scores **0.622**, a gain of +0.066, as large as on all builds, with every fold's interval above zero.
- With a 15% review budget, Tier E catches 30% of new breakages (lift 1.78×) against 24% for Tier C.

The honest headline for *fresh* breakages is therefore about 0.62, not 0.70. This analysis was carried out after the main results (post hoc).

8.5 Pre-Registered Tests on GitHub Actions

The frozen recipe was tested on GitHub Actions, a different CI platform, without any changes.

Test 1 (30 repositories): inconclusive

Tier E scored 0.770 against 0.715 (+0.055) and was ahead in all five folds. However, only 3 of 5 fold intervals excluded zero, and the rule required 4, so the pre-registered verdict is *inconclusive* (Table 8.5). Recall@15% was 0.48 against 0.37.

Table 8.5: First GitHub Actions test (30 repositories, folds 6–10)

Setting	Tier C	Tier E	Gain	Folds clearly ahead
All runs (primary)	0.715	**0.770**	+0.055	3/5
Label-validity check	0.703	0.754	+0.051	3/5
New breakages	0.645	0.695	+0.050	0/5
All runs, 6 h buffer	0.698	0.718	+0.021	0/5

Confirmation (30 new repositories): confirms

A second protocol was fixed before collecting data from 30 repositories never used before. Here Tier E scored **0.911** against 0.763 (+0.149). All five fold intervals exclude zero, and the smallest

lower bound is +0.090 (Fig 18, Table 8.6). Recall@15% rose from 0.31 to 0.43, and Tier E beat Tier C in all 24 test repositories. A secondary analysis on both datasets together (60 repositories) gave +0.179, again 5 of 5 folds.

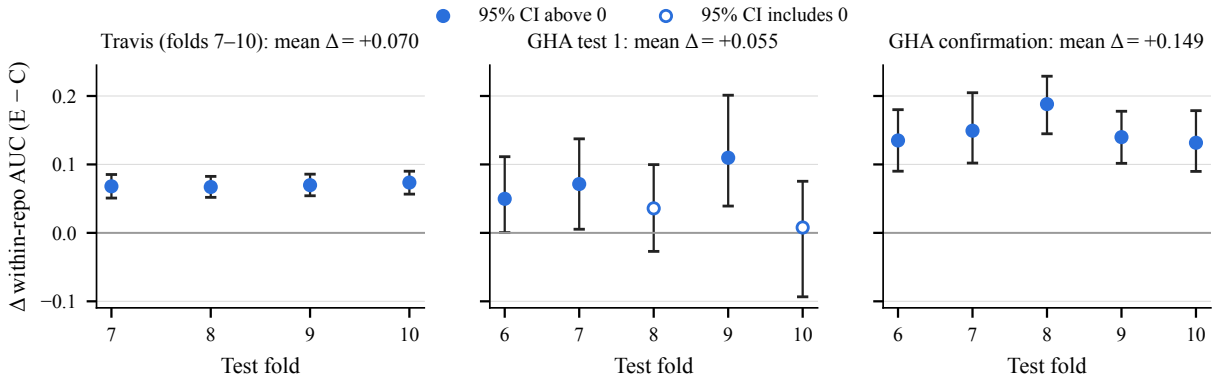


Fig 18: Per-fold gain of Tier E over Tier C with 95% confidence intervals. Filled markers: the interval excludes zero.

Table 8.6: GitHub Actions confirmation test (30 new repositories, folds 6–10)

Setting	Tier C	Tier E	Gain	Folds clearly ahead
All runs (primary)	0.763	**0.911**	+0.149	5/5
Label-validity check	0.765	0.911	+0.147	5/5
New breakages	0.642	0.683	+0.041	2/5
All runs, 6 h buffer	0.741	0.846	+0.105	5/5
New breakages, 6 h buffer	0.637	0.634	−0.004	0/5
Combined, 60 repositories	0.685	0.864	+0.179	5/5

Is it only “it failed last time”?

The confirmation gain is almost three times the first test’s, and failures were three times as common. We therefore checked a naive rule that predicts failure when the stream’s previous finished run failed. It scores only about 0.72, below Tier C (Fig 19), and the gain per repository is unrelated to the repository’s failure rate (correlation 0.03). This check was exploratory.

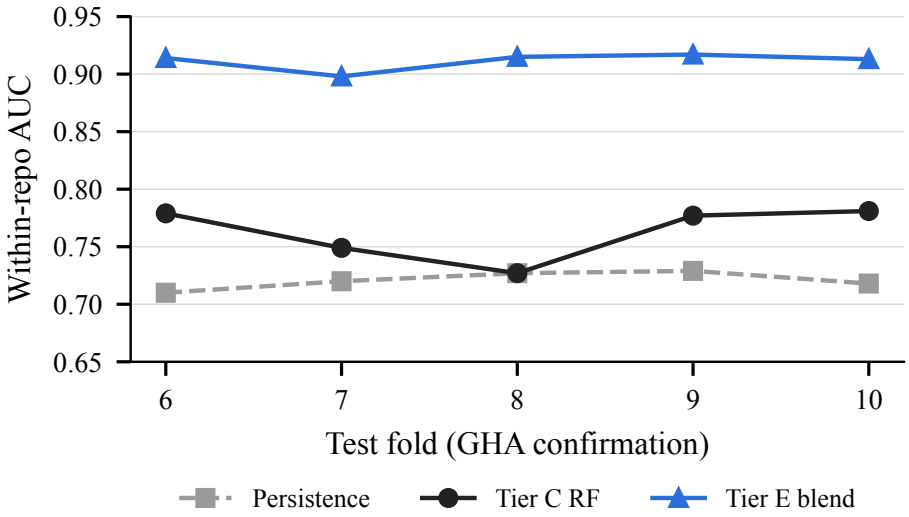


Fig 19: Confirmation test: Tier E against Tier C and a naive persistence rule

Discussion

On GitHub Actions, the stream and in-flight features carry the gain; the ranking objective alone adds only +0.010 (test 1) and +0.042 (confirmation). Most runs finish within an hour, so the very recent outcome of the same stream is the key signal. Withholding it for 6 hours removes most of the gain on new breakages. The size of the gain varies by project (+0.055 to +0.149), but its direction is consistent across Travis CI and both GitHub Actions tests.

8.6 Deployed Model and Alert Policy

The deployed model was trained on folds 1–9 of the 146,556 GitHub Actions runs. Table 8.7 shows its results on the newest 10%. To estimate how it performs on a repository it has never seen, the holdout was rescored with all workflow identities hidden; it still reaches 0.893.

Table 8.7: Deployed model on its holdout (newest 10% of GitHub Actions runs, 20 repositories)

Measure	Value
Within-repository AUC	0.913
Within-repository AUC, repository never seen	0.893
Within-repository recall at a 15% review budget	43.3%
Pooled AUC	0.936
Failure rate of the holdout	36.3%

Alert policy

A single global threshold set to flag 15% of the holdout sits at 0.94, because that holdout fails

often. It flags between 3% and 51% of a repository's runs. The per-repository budget instead flags each repository's riskiest 15% of recent runs (Section 7.6). Replayed in time order on the holdout, each repository flags 11–22% of its runs, and the share of a repository's failures caught rises from 34% to 41%, close to the 43% measured in hindsight (Table 8.8, Fig 20). Pooled precision is lower only because the global threshold spends almost all its alerts on the repositories that fail most.

Table 8.8: Alert policies compared on the deployed model's holdout

Measure	**Global threshold (0.94)**	**Per-repository budget**
Runs flagged per repository	3–51%	11–22%
Failures caught per repository (mean)	34%	**41%**
Share of alerts that were failures	0.97	0.80

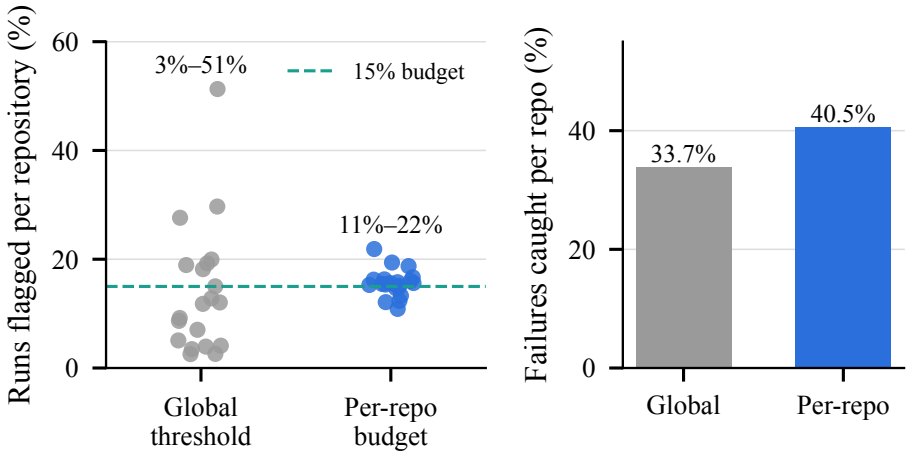


Fig 20: Global threshold vs. per-repository alert budget: share of each repository's runs flagged (left) and failures caught (right)

Public demonstration

In the public demonstration, the 450 replayed runs are scored with the history the system builds up live, starting empty. Within-repository AUCs are 0.80 (freeCodeCamp), 0.95 (bootstrap) and 0.74 (yt-dlp), close to the research evaluation of the same runs (0.78, 0.97 and 0.79). The model-evidence panel of the dashboard (Fig 21) presents these research results alongside the deployed model's holdout statistics.

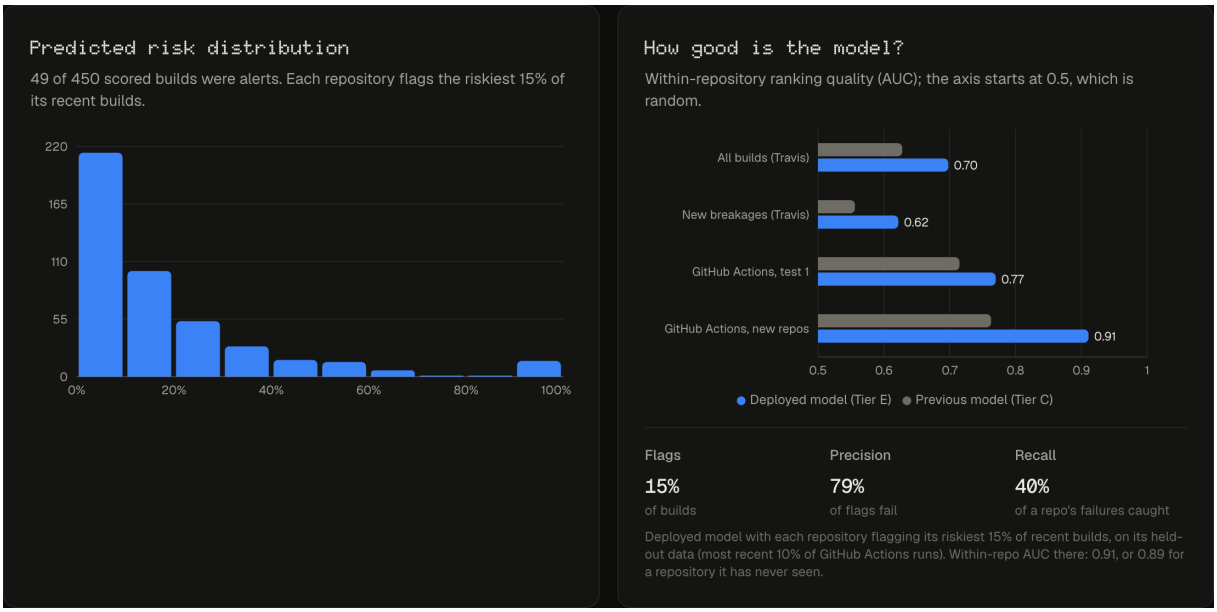


Fig 21: Dashboard: risk distribution of the 450 demonstration runs (49 alerts under the per-repository budget) and the model-evidence panel

8.7 Negative Results

- **Repository-level streaks hurt.** Adding repository-level failure streaks and time gaps to Tier C *reduced* within-repository AUC by 0.019, significantly in 4 of 5 folds. Streaks that mix all of a repository's branches are noise; per-stream history is the version of this idea that helps.

- **Anomaly flag.** Builds flagged as anomalous by Isolation Forest did not fail more often, so the flag is kept as a diagnostic rather than presented as a risk signal.

8.8 Threats to Validity

- **Post-hoc analyses.** The new-breakage analysis and the persistence check were carried out after the main results. On Travis CI fold 6 was used for development, so only folds 7–10 are confirmatory.

- **Approximate finish times.** Finish times on Travis CI are derived from job durations, with a fallback for missing ones.

- **Lost runs.** In the confirmation data, 4.6% of runs were lost to unfetchable fork commits, and these runs failed more often (38.5% vs. 28.2%).

- **Selection.** Repositories come from popular open-source projects. Capping each repository at 4,000 runs left six of the busiest projects in training folds only.

- **What the metric measures.** Within-repository AUC measures ranking, not calibrated

probability. Recall at a 15% budget assumes that teams review a fixed share of builds.

CHAPTER 9 – CONCLUSION AND FUTURE WORK

9.1 Conclusion

CI/CD has made the integration, testing and deployment of code largely automatic. Yet frequent build failures, long execution times, dependency issues and manual debugging remain challenges, and current CI/CD systems concentrate on automation and monitoring rather than failure prediction and optimization.

This project designed, built and evaluated an AI-based system that predicts CI/CD build failures before a build runs, detects bottlenecks, groups and diagnoses failures, and presents the results on a dashboard. The main conclusions are:

- **Evaluation matters as much as the model.** The same kind of classifier scores 0.91 AUC on a random split but only 0.63 under a protocol that matches real use. That protocol uses only information available at trigger time, counts a past build only once it has finished, evaluates chronologically, and measures ranking within each repository. Results that skip these steps should not be trusted.

- **Stream-aware ranking works.** Adding the history of the build's own branch or pull request, the state of builds still running, and a learning-to-rank objective improved within-repository AUC on Travis CI from 0.628 to 0.698. It also improved from 0.556 to 0.622 on new breakages, where the last outcome gives no clue.

- **The result carries over.** Tested on GitHub Actions under protocols fixed in advance, the gain was positive in every fold. The first test was inconclusive under its strict rule (3 of 5 folds); a confirmation on 30 new repositories met the rule in all five folds (0.911 vs. 0.763).

- **The system is usable.** The deployed service scores GitHub Actions runs automatically and explains each prediction. Each repository reviews its riskiest 15% of runs, which catches about 41% of its failures. A read-only public demonstration shows the system on real, out-of-sample runs.

Objectives achieved

1. *Predict build failures from historical data.* Achieved, with a leakage-safe protocol and pre-registered external confirmation (Sections 8.3–8.5).

2. *Detect bottlenecks and abnormal execution.* Achieved: job durations are compared with their own history to flag slow and stalled jobs, and Isolation Forest flags unusual inputs

(Section 7.5).

3. *Automated root cause analysis and suggestions on a dashboard.* Achieved: failure logs are clustered and diagnosed, each category maps to a suggestion, and everything is presented on the dashboard (Sections 7.5 and 7.7).

9.2 Future Work

- **Forward-looking test.** A test of the frozen deployed system has been pre-registered. It will score October 2026 runs from the 60 repositories, collected from 28 November 2026, with the model and alert rule fixed by their SHA-256 fingerprints. It counts as holding up if the model clearly beats a "last run failed" baseline and reaches at least 0.80 within-repository AUC.

- **Earlier warning for first failures.** Gains on new breakages are smaller and vanish when outcomes are delayed. Richer change information, such as which files and tests a change touches, may help.

- **More CI platforms.** Collect and evaluate data from GitLab CI/CD and Jenkins, beyond the existing Jenkins endpoint.

- **Large-scale deployment.** Run the service on Kubernetes with a production database, and add per-team configuration of the review budget.

- **Deeper log analysis.** Use natural language processing models for log understanding and root cause identification, and explore automated self-repair for common failure types.

- **Calibrated probabilities per repository**, so that a predicted risk can be read as a real chance of failure.

These enhancements would further improve the CI/CD optimization system in effectiveness and reach.

CHAPTER 10 – REFERENCES

[1] C. Zhang, B. Chen, X. Peng, and W. Zhao, "BuildSheriff: Change-aware test failure triage for continuous integration builds," in *Proc. 44th Int. Conf. Software Engineering (ICSE)*, 2022. [Online]. Available: https://chenbihuan.github.io/paper/icse22-zhang-buildsheriff.pdf

[2] I. Saidani, A. Ouni, M. Chouchen, and M. W. Mkaouer, "Predicting continuous integration build failures using evolutionary search," *Information and Software Technology*, 2020. [Online]. Available: https://ouniali.github.io/papers/IST2020-saidani.pdf

[3] A. Barrak, E. E. Eghan, B. Adams, and F. Khomh, "Why do builds fail? A conceptual replication study," *Journal of Systems and Software*, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0164121221000364

[4] T. Rausch, W. Hummer, P. Leitner, and S. Schulte, "An empirical analysis of build failures in the continuous integration workflows of Java-based open-source software," in *Proc. 14th Int. Conf. Mining Software Repositories (MSR)*, 2017, pp. 345–355. [Online]. Available: https://ieeexplore.ieee.org/document/7962384

[5] T. Durieux, C. Le Goues, M. Hilton, and R. Abreu, "Empirical study of restarted and flaky builds on Travis CI," in *Proc. 17th Int. Conf. Mining Software Repositories (MSR)*, 2020. [Online]. Available: https://2020.msrconf.org/details/msr-2020-papers/21

[6] T. A. Ghaleb, D. A. da Costa, and Y. Zou, "An empirical study of the long duration of continuous integration builds," *Empirical Software Engineering*, 2019. [Online]. Available: https://link.springer.com/article/10.1007/s10664-019-09695-9

[7] I. Saidani, A. Ouni, and M. W. Mkaouer, "Improving the prediction of continuous integration build failures using deep learning," *Automated Software Engineering*, 2022. [Online]. Available: https://www.researchgate.net/publication/357990835

[8] R. Farihane, "CI/CD pipeline optimization using AI: A systematic mapping study," *MDPI Proceedings*, 2025.

[9] R. Kumar, D. N. D. Kollipara, A. H. Mohammed, S. Kumar, F. B. Mohammed, and A. Kumar, "Optimizing CI/CD pipelines with AI-driven build failure prediction: An empirical study on machine learning models for early failure detection," 2025.

[10] Y. Hong, C. Tantithamthavorn, J. Pasuksmit, P. Thongtanunam, A. Friedman, X. Zhao, and A. Krasikov, "Practitioners' challenges and perceptions of CI build failure predictions at Atlassian," arXiv preprint arXiv:2402.09651, 2024.

[11] M. Hilton, T. Tunnell, K. Huang, D. Marinov, and D. Dig, "Usage, costs, and benefits of continuous integration in open-source projects," in *Proc. 31st IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2016, pp. 426–437.

[12] M. Beller, G. Gousios, and A. Zaidman, "TravisTorrent: Synthesizing Travis CI and GitHub for full-stack research on continuous integration," in *Proc. 14th Int. Conf. Mining Software Repositories (MSR)*, 2017, pp. 447–450.

[13] F. Hassan and X. Wang, "Change-aware build prediction model for stall avoidance in continuous integration," in *Proc. ACM/IEEE Int. Symp. Empirical Software Engineering and Measurement (ESEM)*, 2017, pp. 157–162.

[14] A. Ni and M. Li, "Cost-effective build outcome prediction using cascaded classifiers," in *Proc. 14th Int. Conf. Mining Software Repositories (MSR)*, 2017, pp. 455–458.

[15] J. Xia and Y. Li, "Could we predict the result of a continuous integration build? An empirical study," in *Proc. IEEE Int. Conf. Software Quality, Reliability and Security Companion (QRS-C)*, 2017, pp. 311–315.

[16] B. Chen, L. Chen, C. Zhang, and X. Peng, "BuildFast: History-aware build outcome prediction for fast feedback and reduced cost in continuous integration," in *Proc. 35th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2020, pp. 42–53.

[17] X. Jin and F. Servant, "A cost-efficient approach to building in continuous integration," in *Proc. 42nd ACM/IEEE Int. Conf. Software Engineering (ICSE)*, 2020, pp. 13–25.

[18] K. Gallaba, C. Macho, M. Pinzger, and S. McIntosh, "Noise and heterogeneity in historical build data: An empirical study of Travis CI," in *Proc. 33rd ACM/IEEE Int. Conf. Automated Software Engineering (ASE)*, 2018, pp. 87–97.

[19] T. Durieux, R. Abreu, M. Monperrus, T. F. Bissyandé, and L. Cruz, "An analysis of 35+ million jobs of Travis CI," in *Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME)*, 2019, pp. 291–295.

[20] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 4, 2012.

[21] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Information Sciences*, vol. 191, pp. 192–213, 2012.

[22] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, "The preregistration revolution," *Proc. National Academy of Sciences*, vol. 115, no. 11, pp. 2600–2606, 2018.

[23] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[24] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.

[25] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[26] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[27] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[28] S. Bruch, "An alternative cross entropy loss for learning-to-rank," in *Proc. Web Conference (WWW)*, 2021, pp. 118–126.

[29] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in *Advances in Large Margin Classifiers*. MIT Press, 1999, pp. 61–74.

[30] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[31] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.

[32] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[33] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[34] S. P. Lloyd, "Least squares quantization in PCM," *IEEE Trans. Information Theory*, vol. 28, no. 2, pp. 129–137, 1982.

[35] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

Project resources. Source code, protocols and results: https://github.com/yasaswinipakam/cicd-ai-optimizer. Read-only demonstration: https://cicd-ai-optimizer.onrender.com.